PipeCheck: Specifying and Verifying Microarchitectural Enforcement of Memory Consistency Models

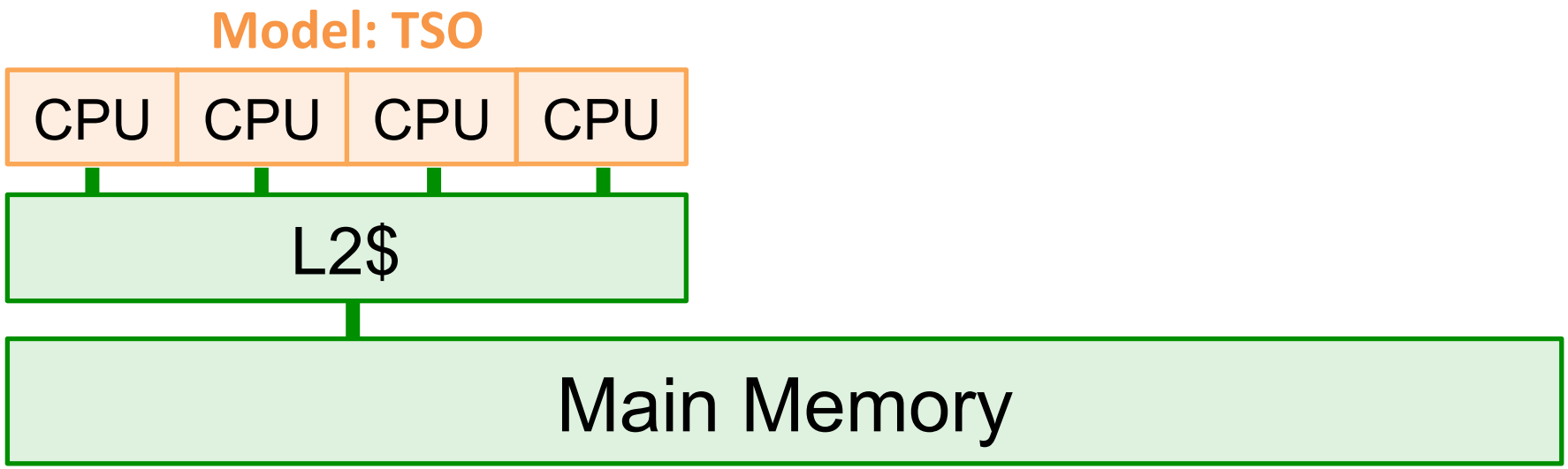
Daniel Lustig[+], Michael Pellauer[*],
and Margaret Martonosi[+]

[+]Princeton University [*]Intel VSSAD

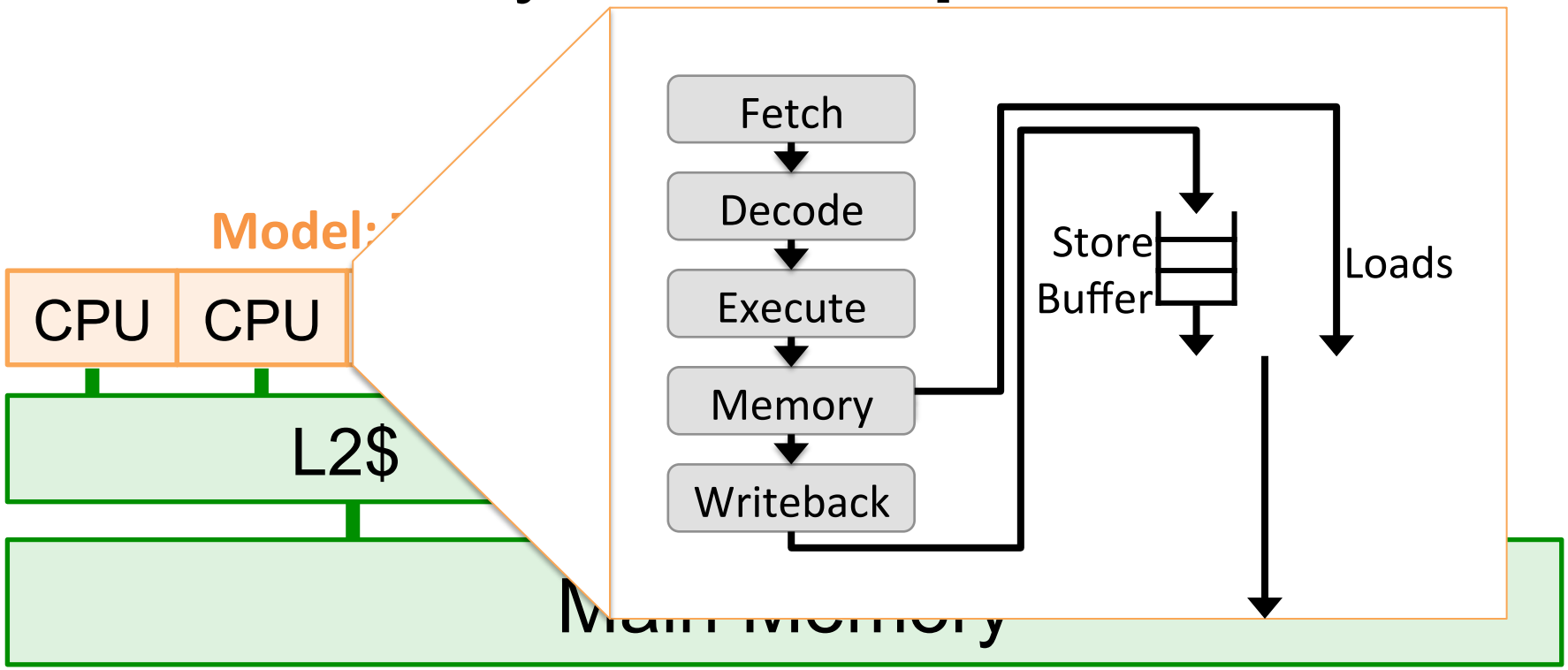
MICRO 47

PRINCETON

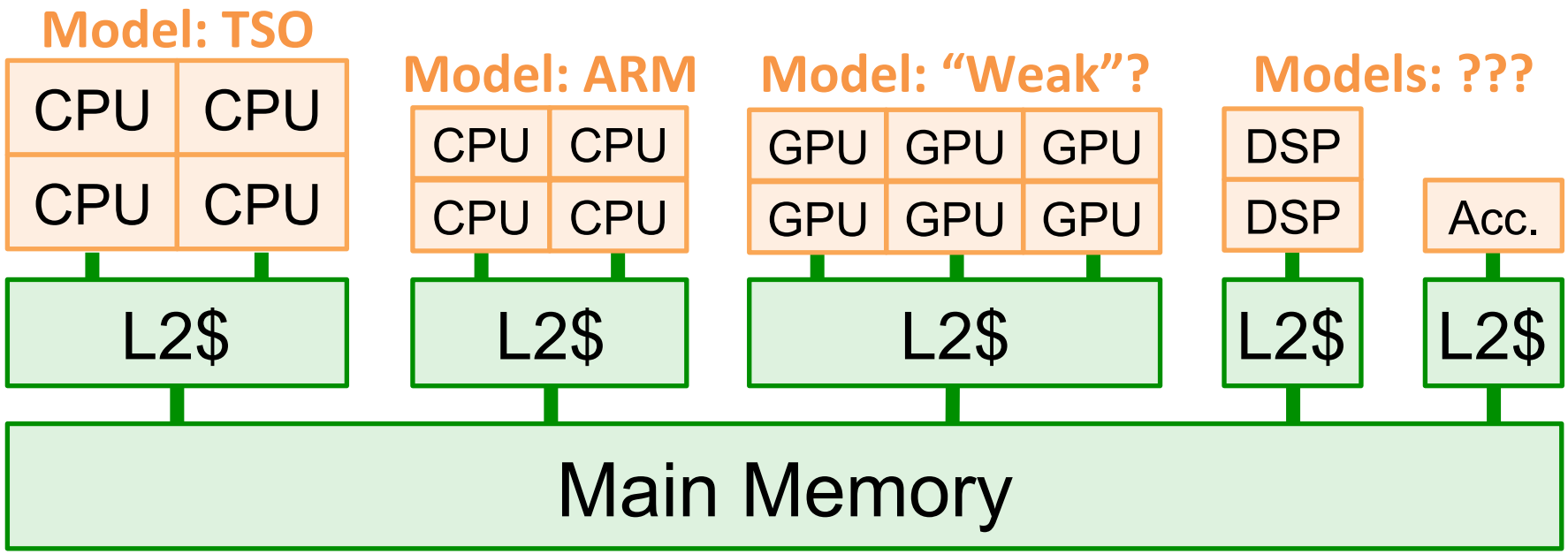
Multicore Systems Need Memory Consistency Models



Motivation: Verify correctness of memory consistency model **implementation**



Motivation: Verify correctness of memory consistency model **implementation**



- Systems are getting more complicated!

Talk Outline

- Architectural Models: Defining Correctness
- PipeCheck µarch-Level Analysis
  - Basic "µarch-happens-before" Graphs
  - Advanced µarch Optimizations
  - Comparing arch and µarch
- Automated Tool and Performance Results
- Conclusions

Memory Models Defined by ISA

8.2.2 Memory Ordering in P6 and More Recent Processor Families

The Intel Core 2 Duo, Intel Atom, Intel Core Duo, Pentium 4, and P6 family processors also use a processor-ordered memory-ordering model that can be further defined as "write ordered with store-buffer forwarding." This model can be characterized as follows.

In a single-processor system for memory regions defined as write-back cacheable, the memory-ordering model respects the following principles (**Note** the memory-ordering principles for single-processor and multiple-processor systems are written from the perspective of software executing on the processor, where the term "processor" refers to a logical processor. For example, a physical processor supporting multiple cores and/or HyperThreading Technology is treated as a multi-processor systems.):

- Reads are not reordered with other reads.
- Writes are not reordered with older reads.
- Writes to memory are not reordered with other writes, with the following exceptions:
  - writes executed with the CLFLUSH instruction;
  - streaming stores (writes) executed with the non-temporal move instructions (MOVNTI, MOVNTQ, MOVNTDQ, MOVNTPS, and MOVNTPD); and
  - string operations (see Section 8.2.4.1).
- Reads may be reordered with older writes to different locations but not with older writes to the same location.

...

Memory Models Defined by ISA

8.2.2 Memory Ordering in P6 and More Recent Processor Families

The Intel Core 2 Duo, Intel Atom, Intel Core Duo, Pentium 4, and P6 family processors also use a processor-ordered memory-ordering model that can be further defined as "write ordered with store-buffer forwarding." This model can be characterized as follows.

In a single-processor system for memory regions defined as write-back cacheable, the memory-ordering model respects the following principles (**Note** the me
processor systems are written from the perspe
"processor" refers to a logical processor. For ex
HyperThreading Technology is treated as a mu

- Reads are not reordered with other reads.
- Writes are not reordered with older reads.
- Writes to memory are not reordered with o
  - writes executed with the CLFLUSH instr
  - streaming stores (writes) executed wit
    MOVNTDQ, MOVNTPS, and MOVNTPD);
  - string operations (see Section 8.2.4.1).
- Reads may be reordered with older writes to different locations but not with older writes to the same location.

Type A (first)

Type B (second)

	Load	Store
Load	No	No
Store	Yes	No

...

Lustig, Pellauer, Martonosi, "PipeCheck"

Memory Models Defined by ISA

8.2.2 Memory Ordering in P6 and More Recent Processor Families

The Intel Core 2 Duo, Intel Atom, Intel Core Duo, Pentium 4, and P6 family processors also use a processor-ordered memory-ordering model that can be further defined as "write ordered with store-buffer forwarding." This model can be characterized as follows.

In a single-processor system for memory regions defined as write-back cacheable, the memory-ordering model

e me
erspe
For ex
a mu
ads.
ads.
with o
l instr

— streaming stores (writes) executed wit
MOVNTDQ, MOVNTPS, and MOVNTPD);

— string operations (see Section 8.2.4.1).

- Reads may be reordered with older writes to different locations but not with older writes to the same location.

How to automate verification of microarchitectural implementations?

Type A (first)

Type B (second)

	Load	Store
Load	No	No
Store	Yes	No

What is the Definition of "Correct"?

Preserved Program Order (PPO) for x86-TSO: Reorderings Permitted?

Type A (first) ↑ Type B (second)

	Load	Store
Load	No	No
Store	Yes	No

[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

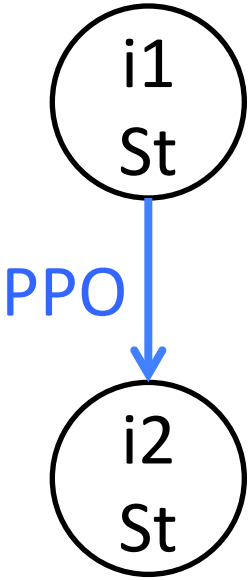
What is the Definition of "Correct"?

Preserved Program Order (PPO) for x86-TSO: Reorderings Permitted?

Type A (first)

Type B (second)

	Load	Store
Load	No	No
Store	Yes	**No**



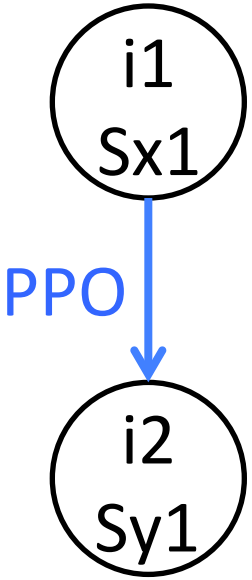
[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

What is the Definition of "Correct"?

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], 1 (i2) st [y], 1	(i3) ld [y] → r1 (i4) ld [x] → r2

Can core 1 observe (i2) before it observes (i1)?



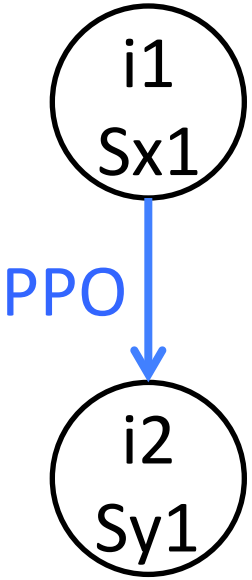
[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

What is the Definition of "Correct"?

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], 1	(i3) ld [y] → r1
(i2) st [y], 1	(i4) ld [x] → r2

Can core 1 observe (i2) before it observes (i1)?



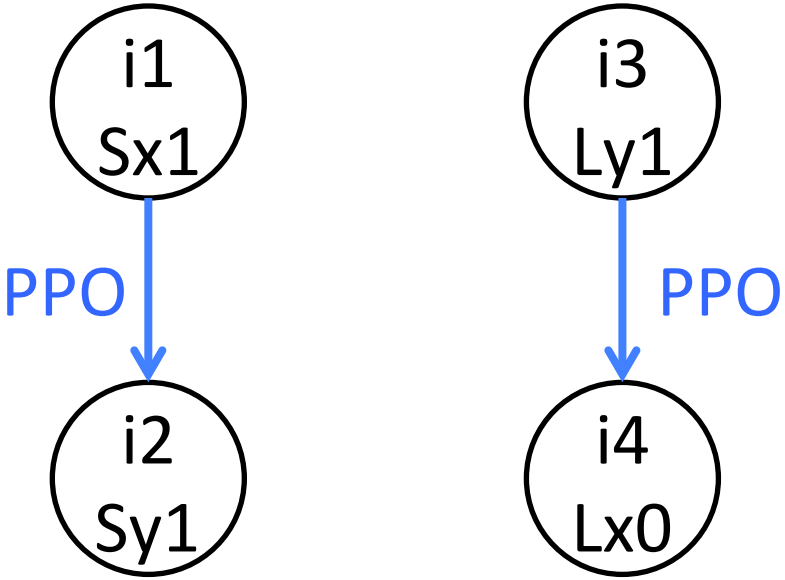
[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

What is the Definition of "Correct"?

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], 1	(i3) ld [y] → r1
(i2) st [y], 1	(i4) ld [x] → r2

Can core 1 observe (i2) before it observes (i1)?



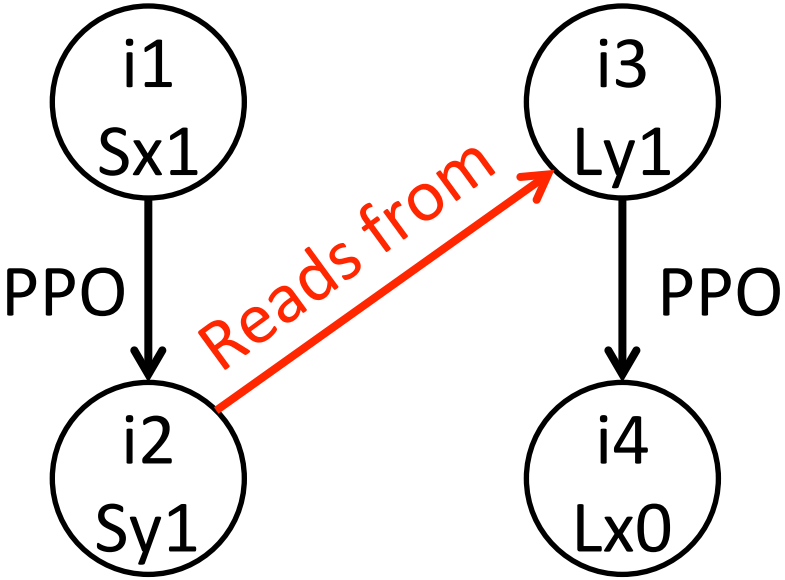
[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

What is the Definition of "Correct"?

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], 1 (i2) st [y], **1**	(i3) ld [y] → **r1** (i4) ld [x] → r2

r1=1



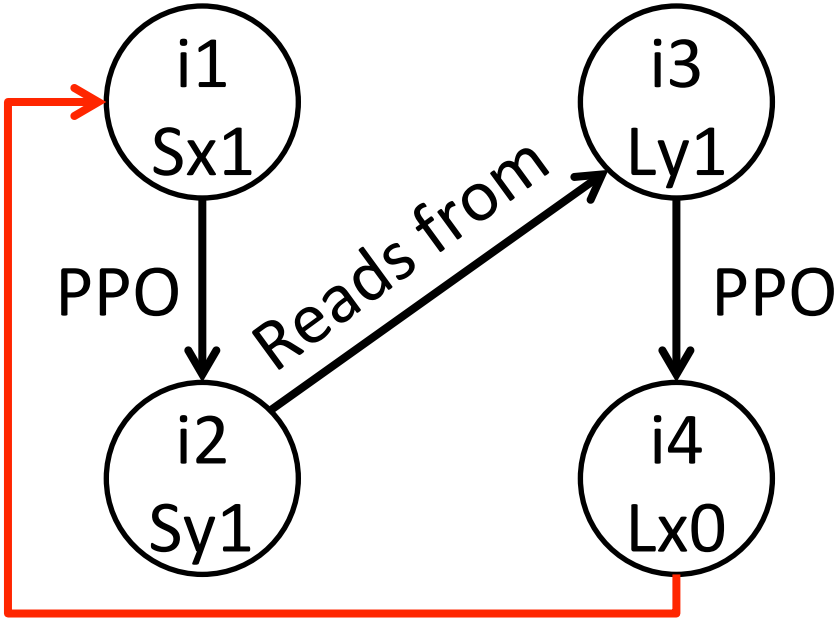
[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

What is the Definition of "Correct"?

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], **1** (i2) st [y], 1	(i3) ld [y] → r1 (i4) ld [x] → **r2**

r1=1, **r2=0**



Reads from earlier store ("from-reads")

[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

What is the Definition of "Correct"?

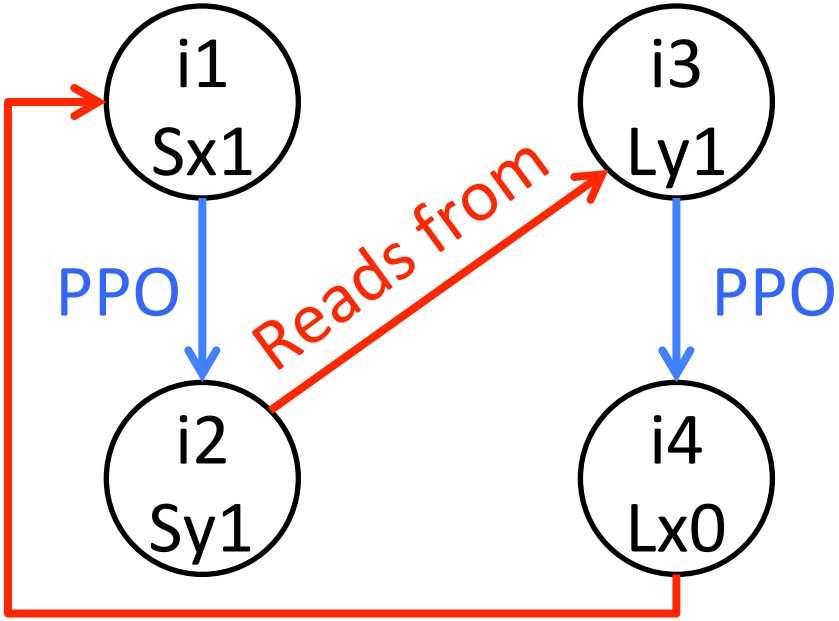
Litmus test mp:

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], 1 (i2) st [y], 1	(i3) ld [y] → r1 (i4) ld [x] → r2

TSO: **Forbid**: r1=1, r2=0

Cycle implies forbidden execution: instruction can't happen before itself

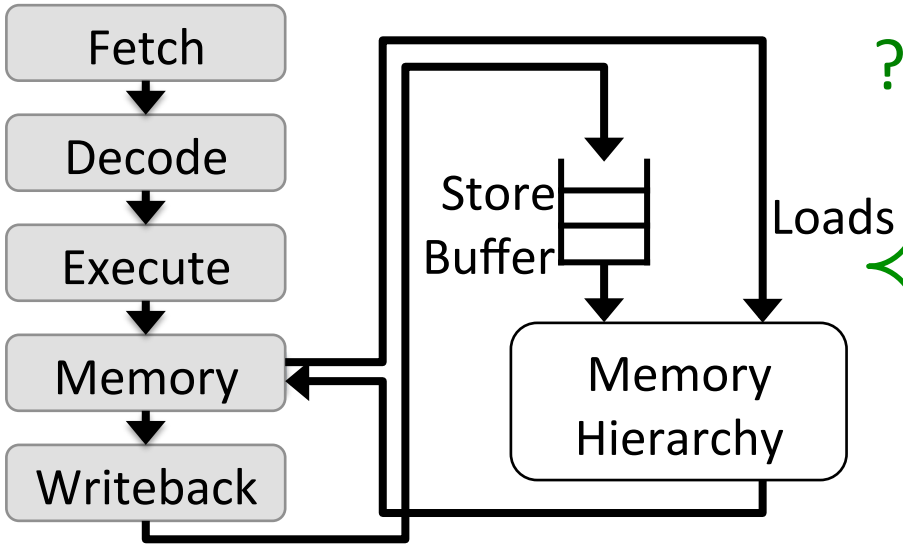


Reads from earlier store ("from-reads")

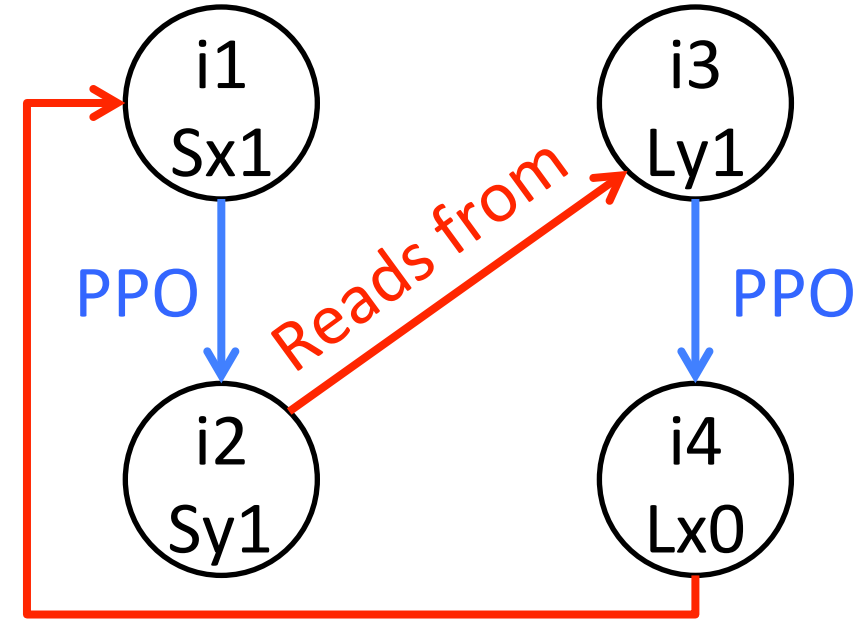
[MD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

Is the *Implementation* Correct?

How is the consistency model **enforced at the microarchitecture level**?



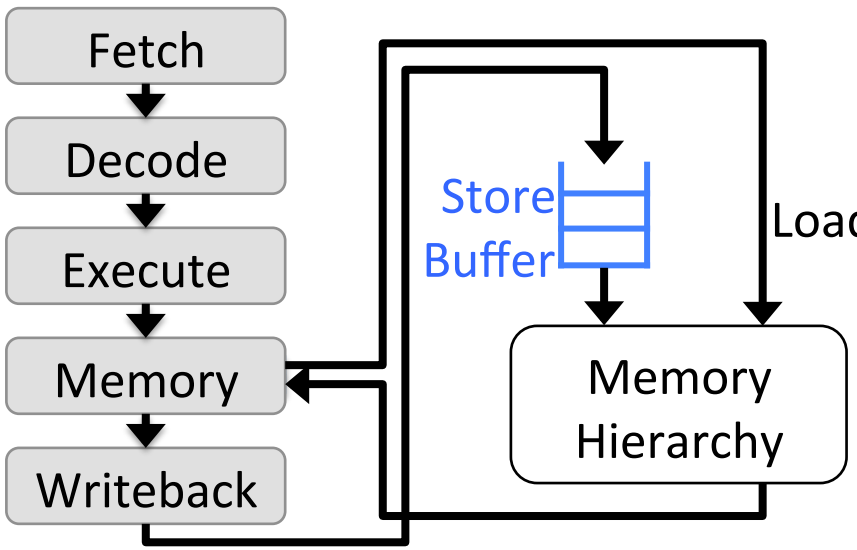
?



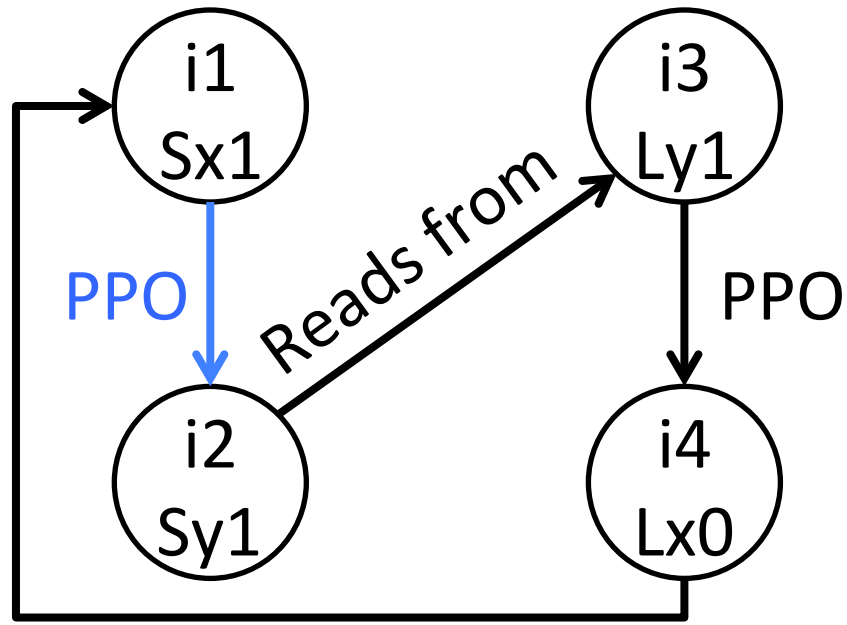
Reads from earlier store
("from-reads")

Is the *Implementation* Correct?

How is the consistency model **enforced at the microarchitecture level**?



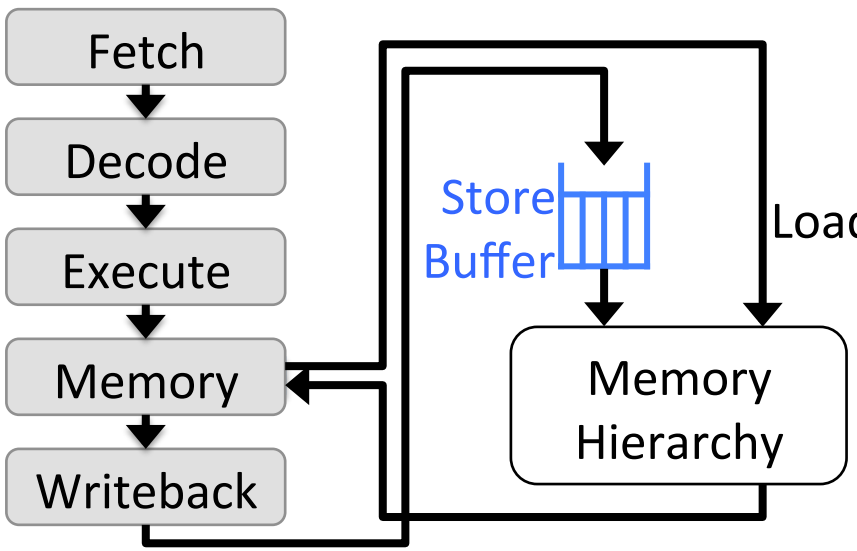
?



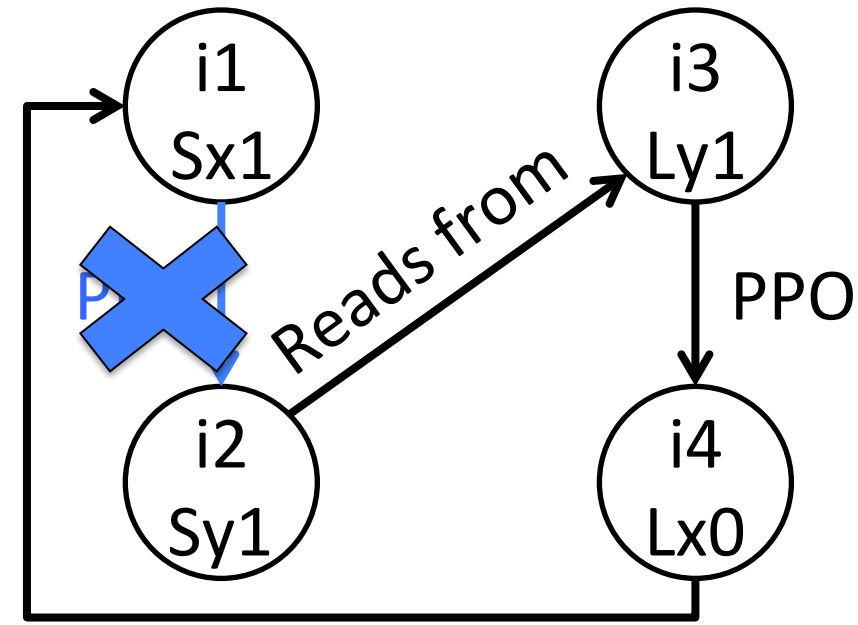
Reads from earlier store ("from-reads")

Is the *Implementation* Correct?

How is the consistency model **enforced at the microarchitecture level**?



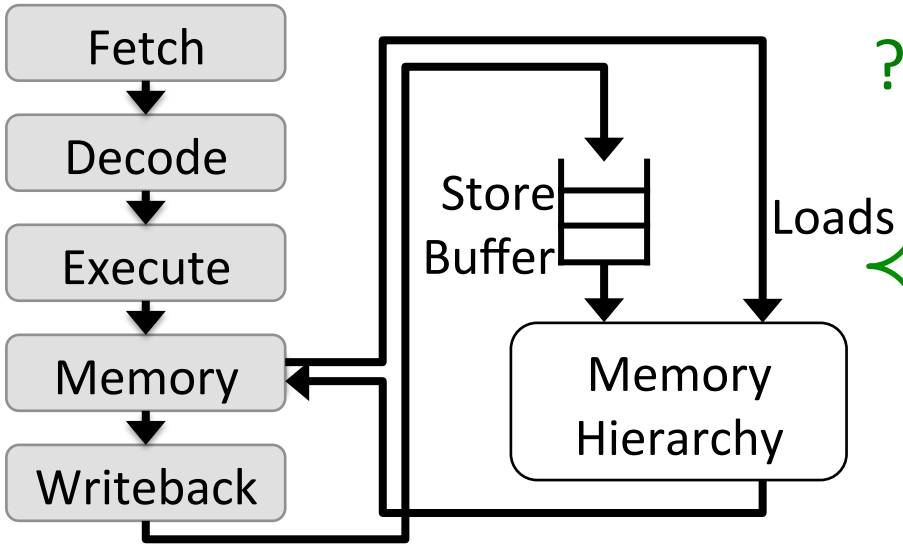
?



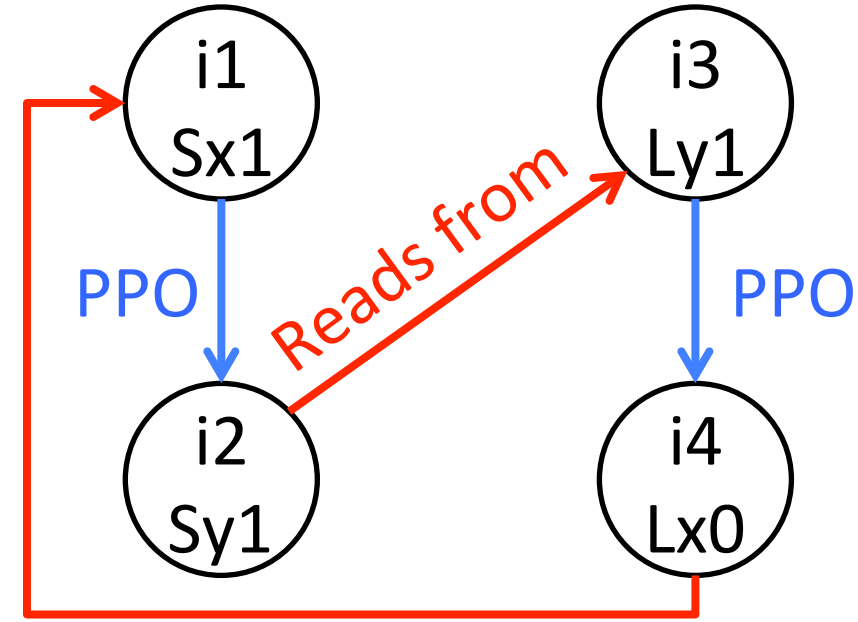
Reads from earlier store ("from-reads")

Is the *Implementation* Correct?

How is the consistency model **enforced at the microarchitecture level**?

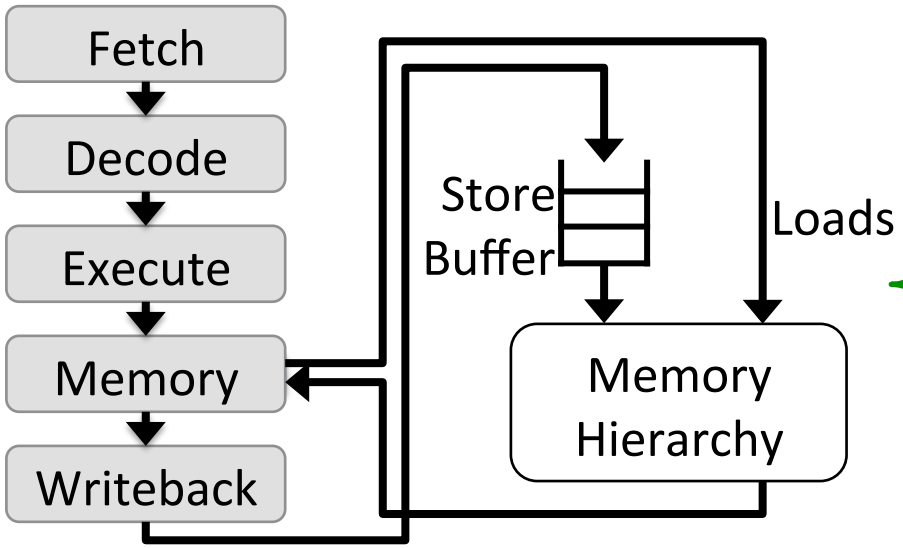


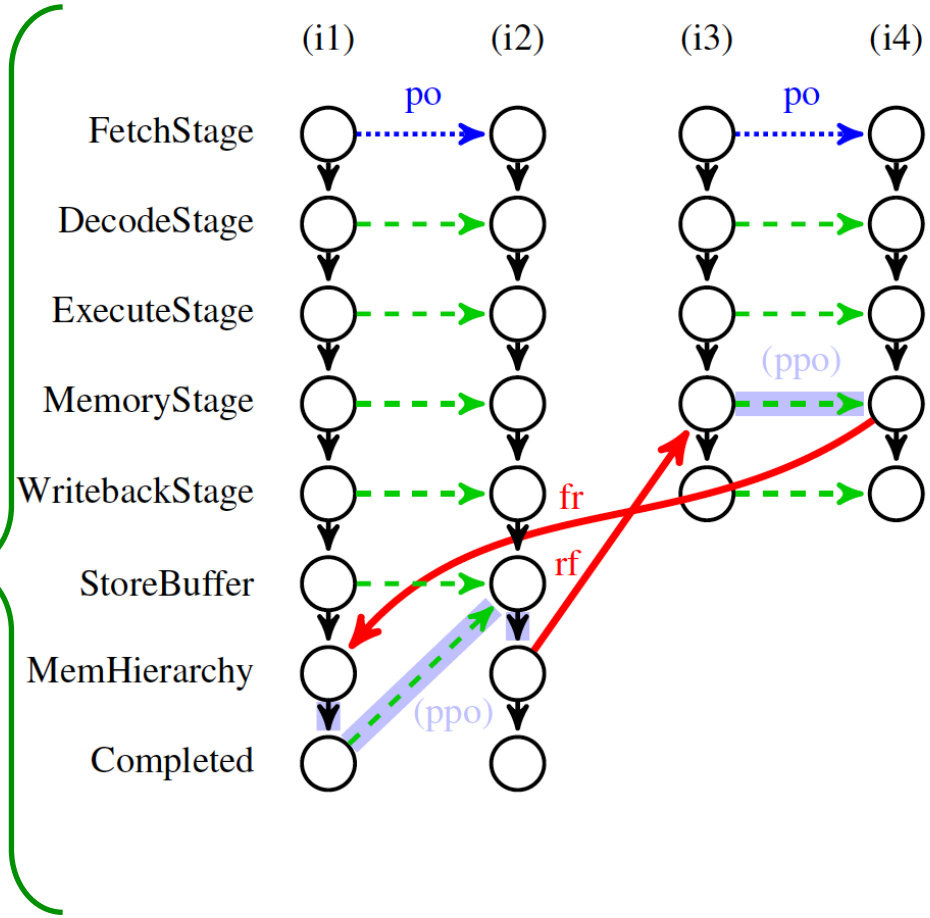
?



Reads from earlier store
("from-reads")

Microarchitectural Happens-Before Graphs

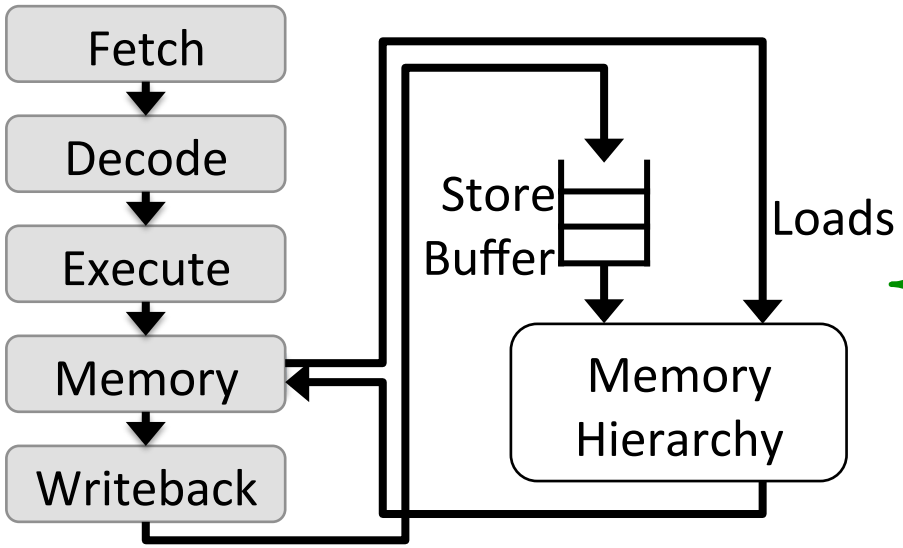


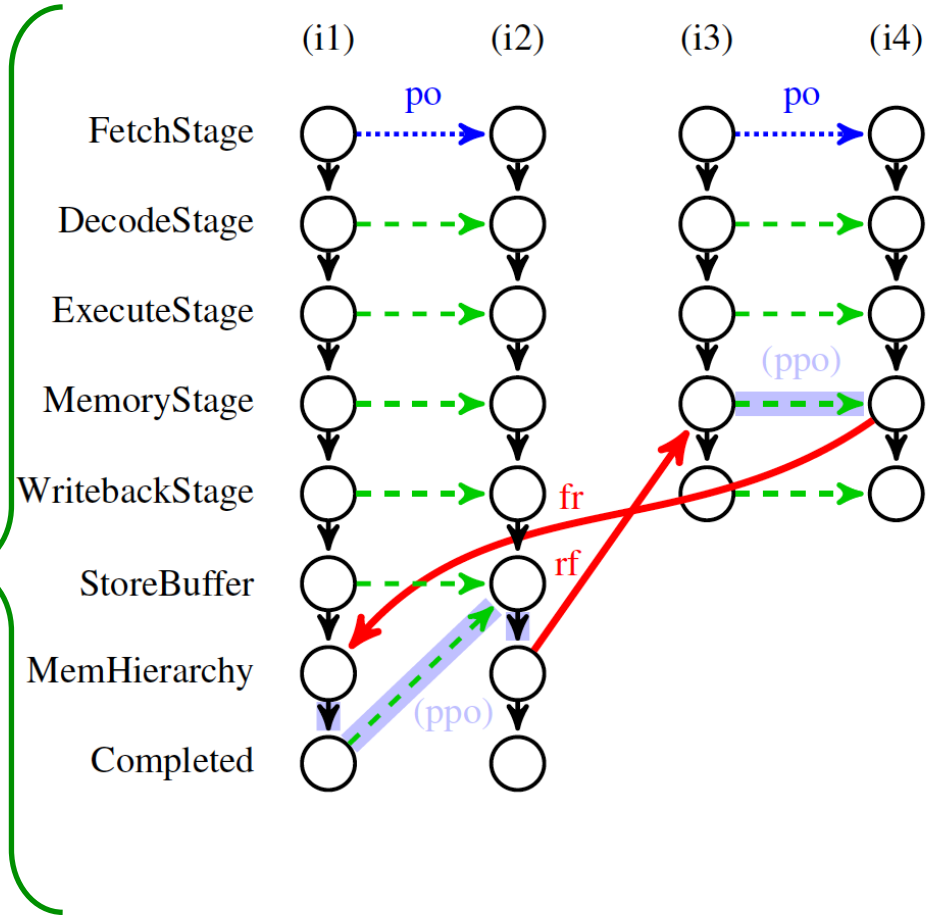


PipeCheck Overview

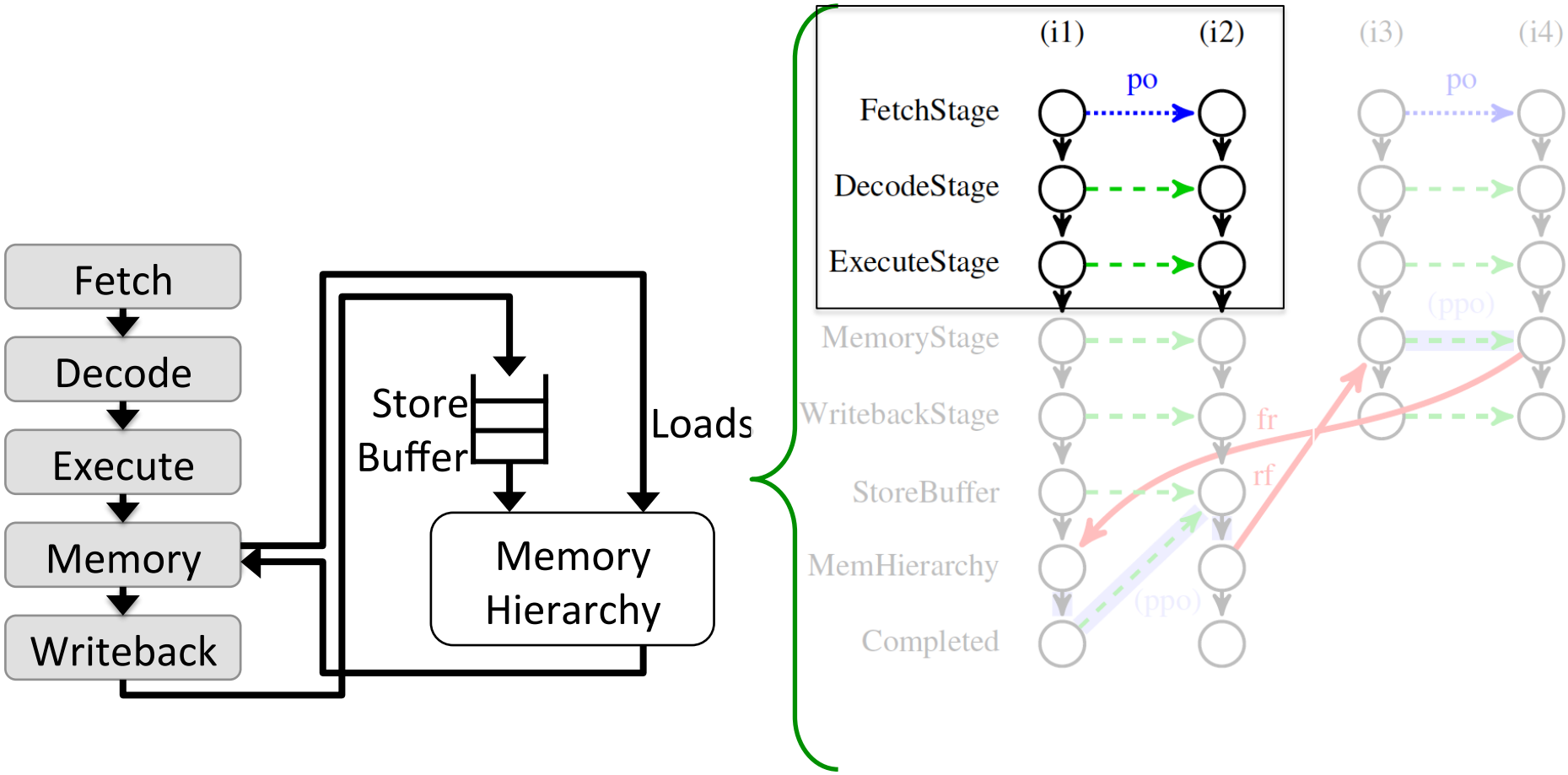
- Key idea: building ***microarchitectural*** happens-before graphs
- Inputs: arch. spec, µarch. spec, test suite
  - PPO Tests
  - Litmus Tests
- Approach: exhaustively enumerate and check all µhb graphs for cycles
  - Quick runtimes due to modest graph sizes

Microarchitectural Happens-Before Graphs



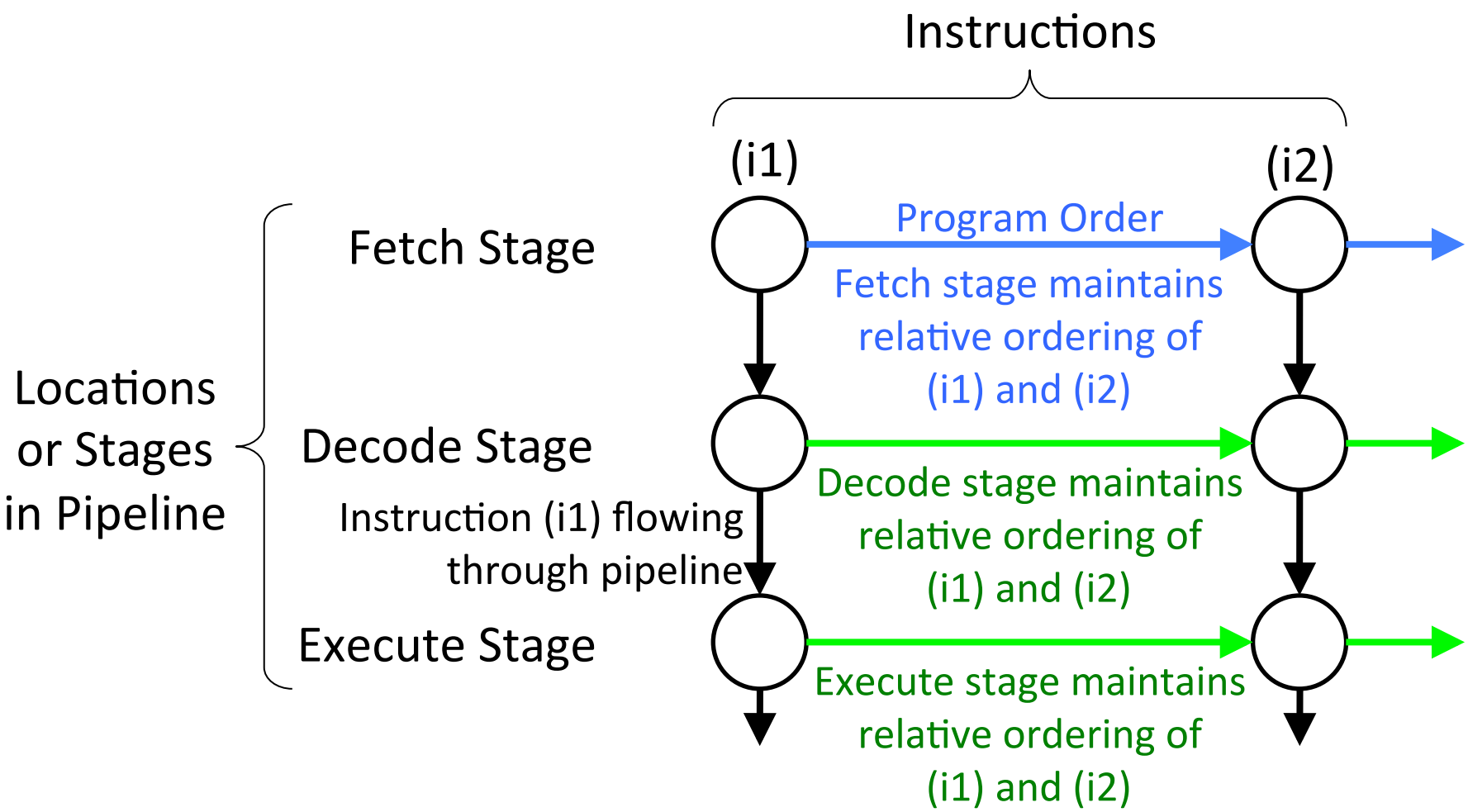


Microarchitectural Happens-Before Graphs

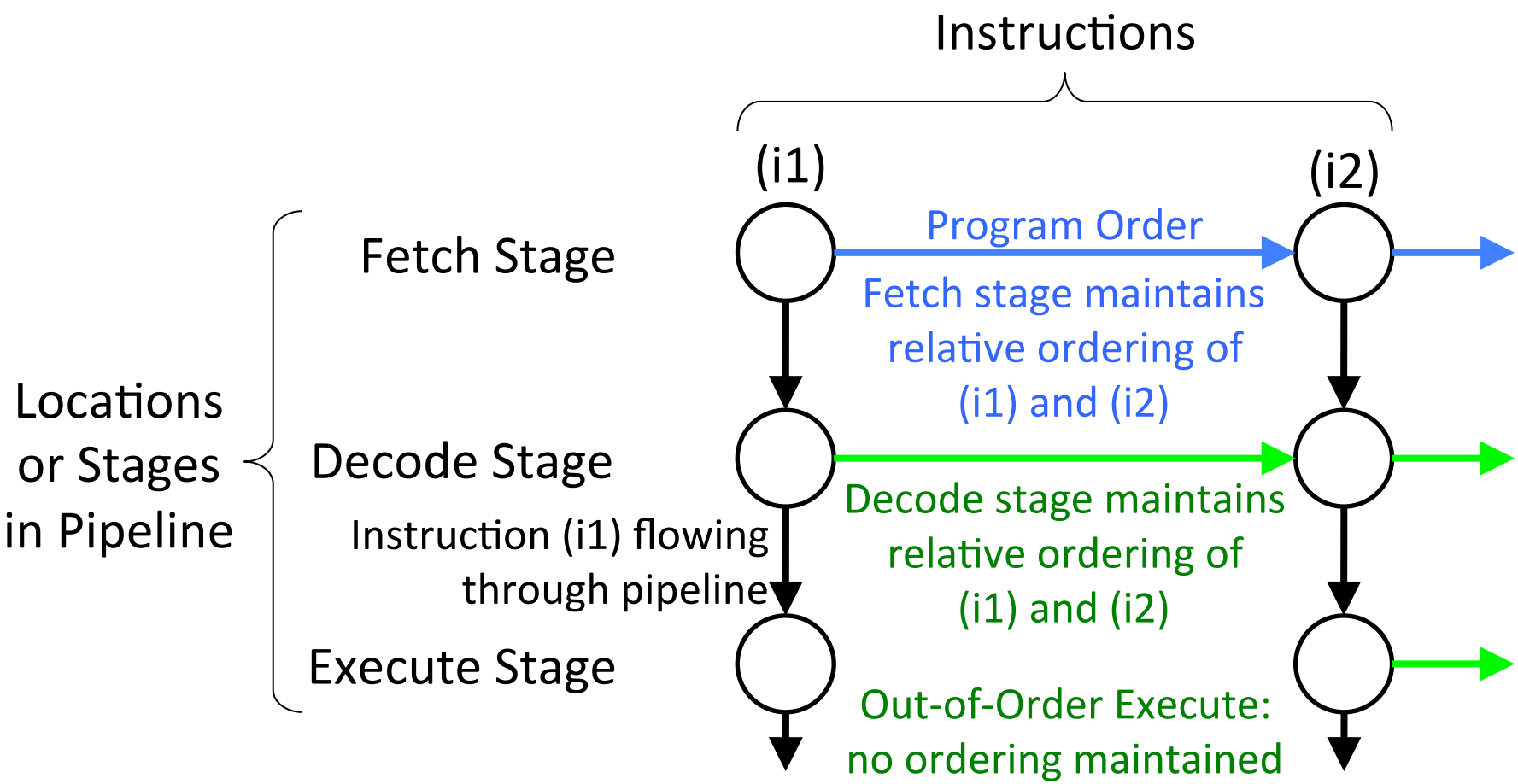


Lustig, Pellauer, Martonosi, “PipeCheck”

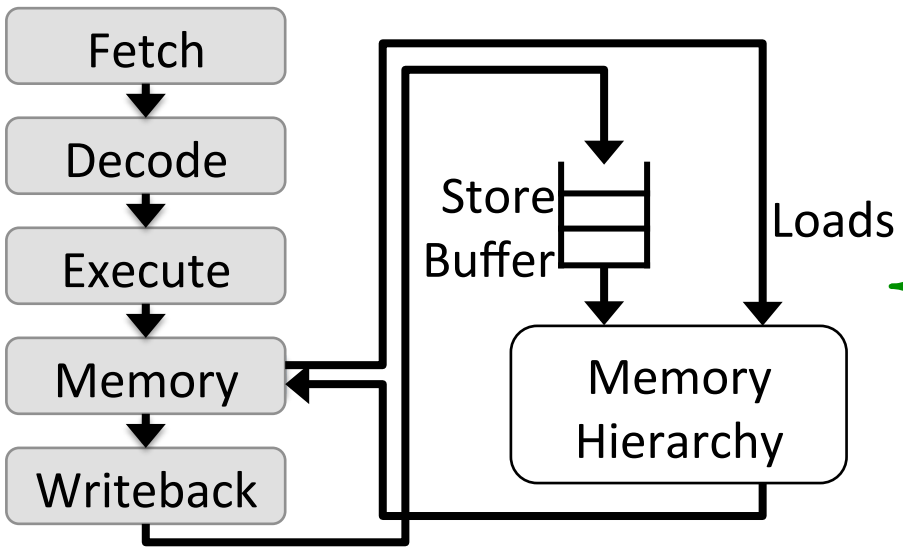
A Closer Look

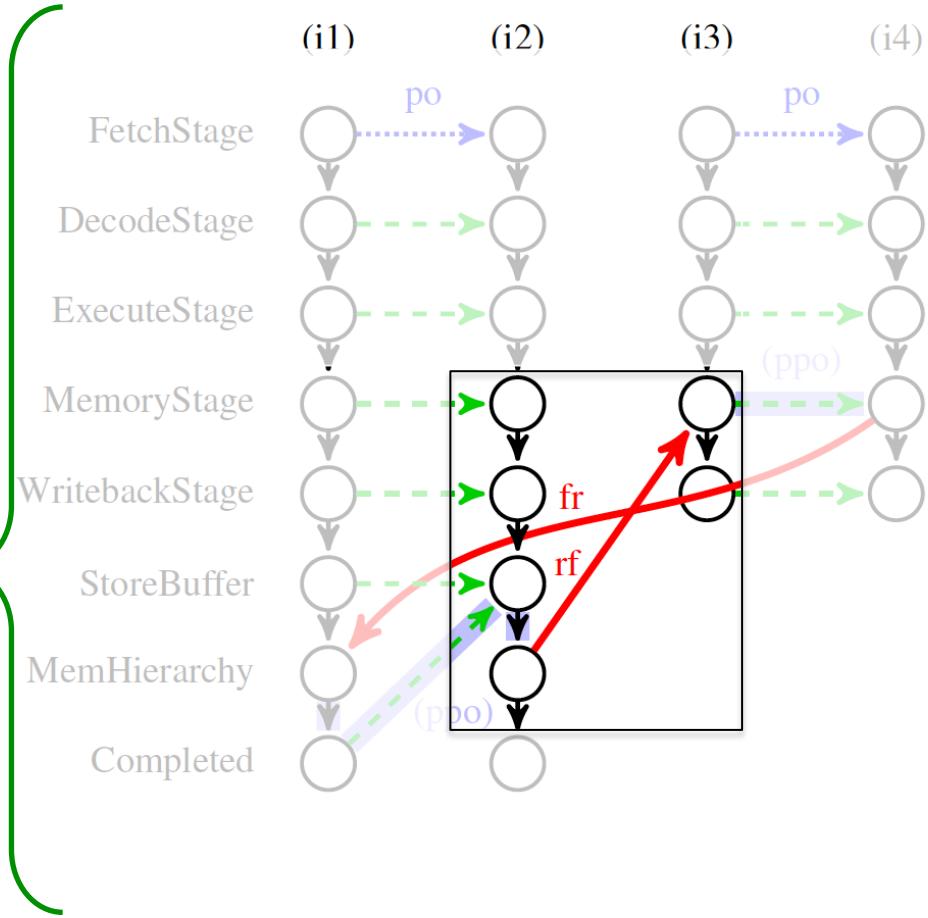


A Closer Look

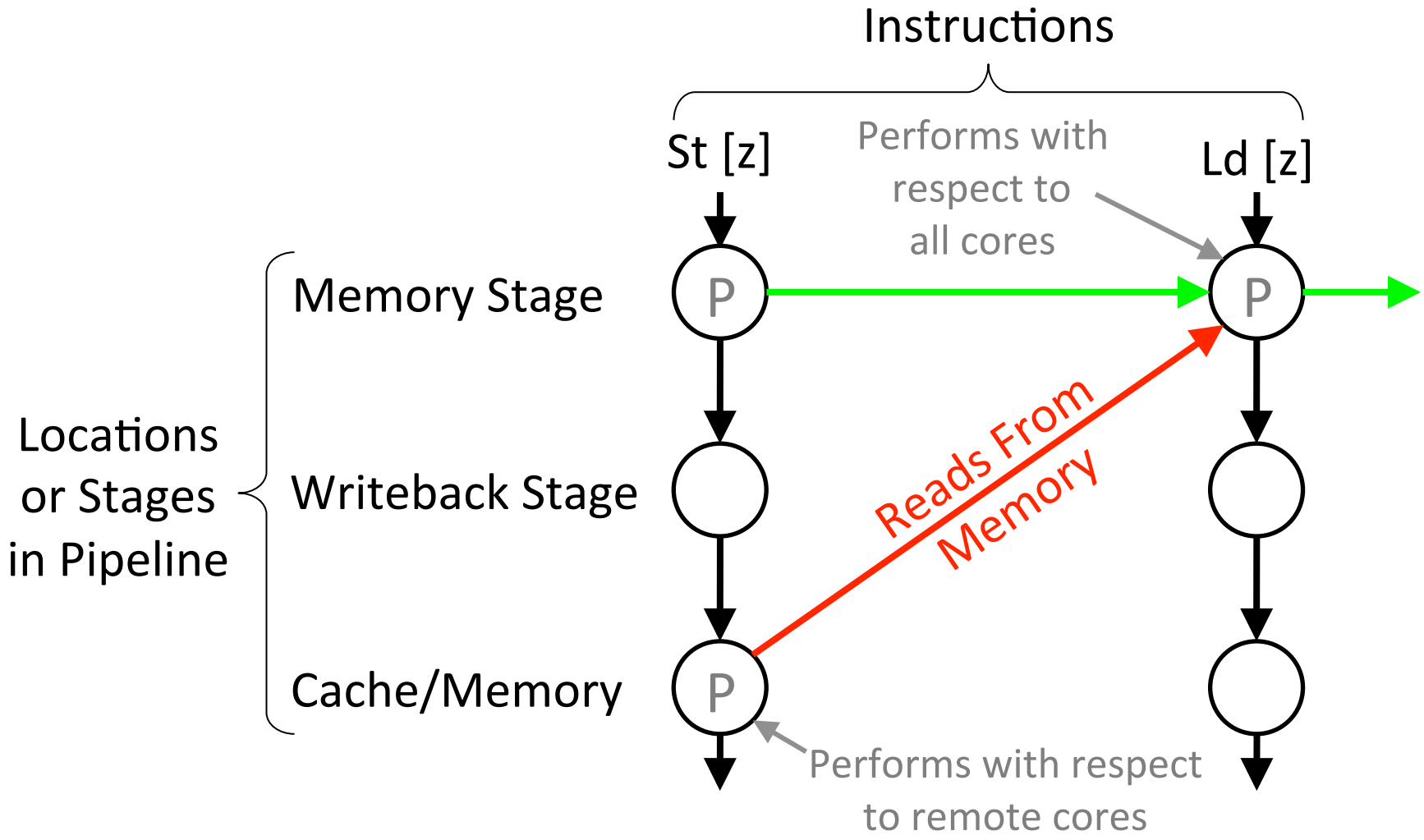


Microarchitectural Happens-Before Graphs

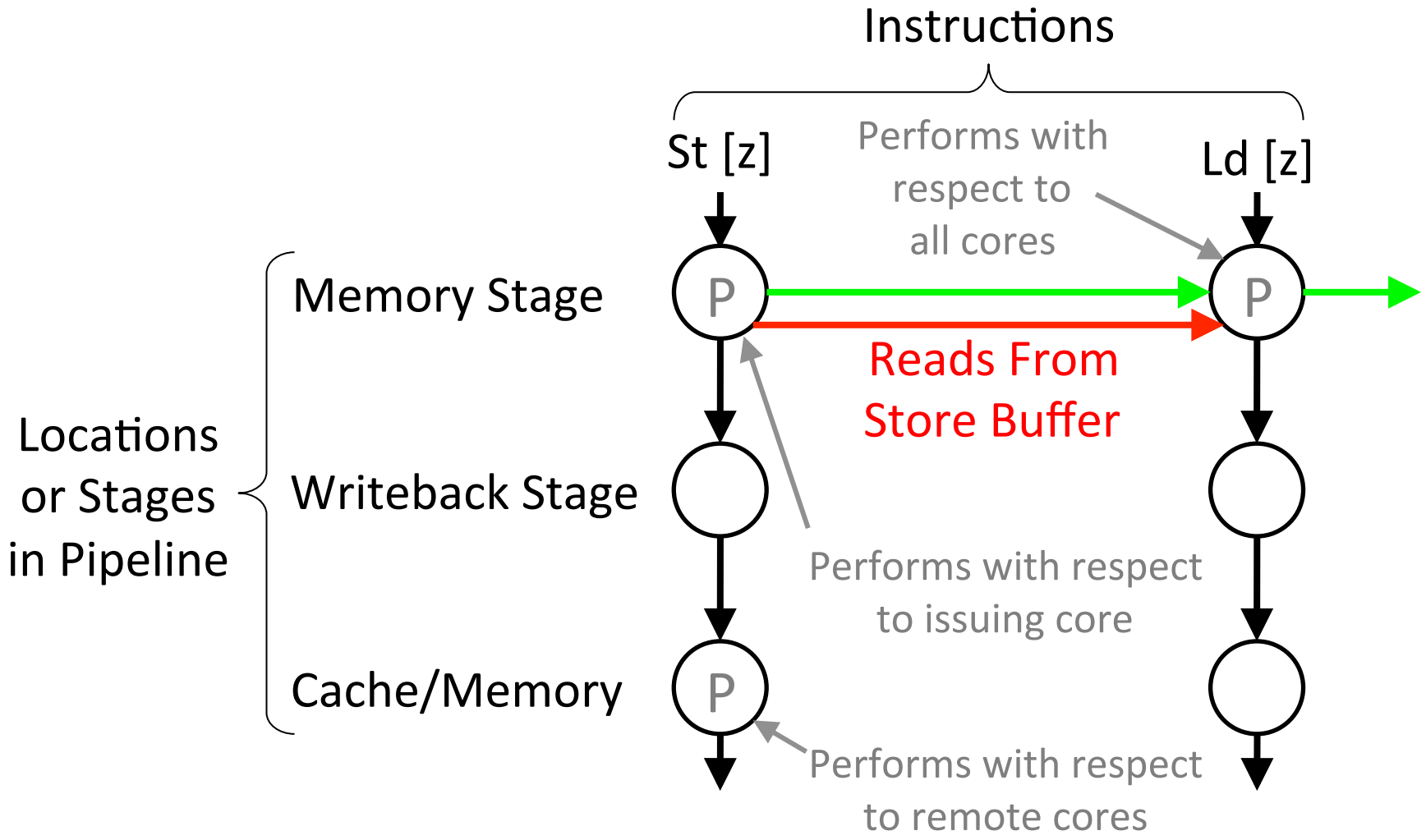




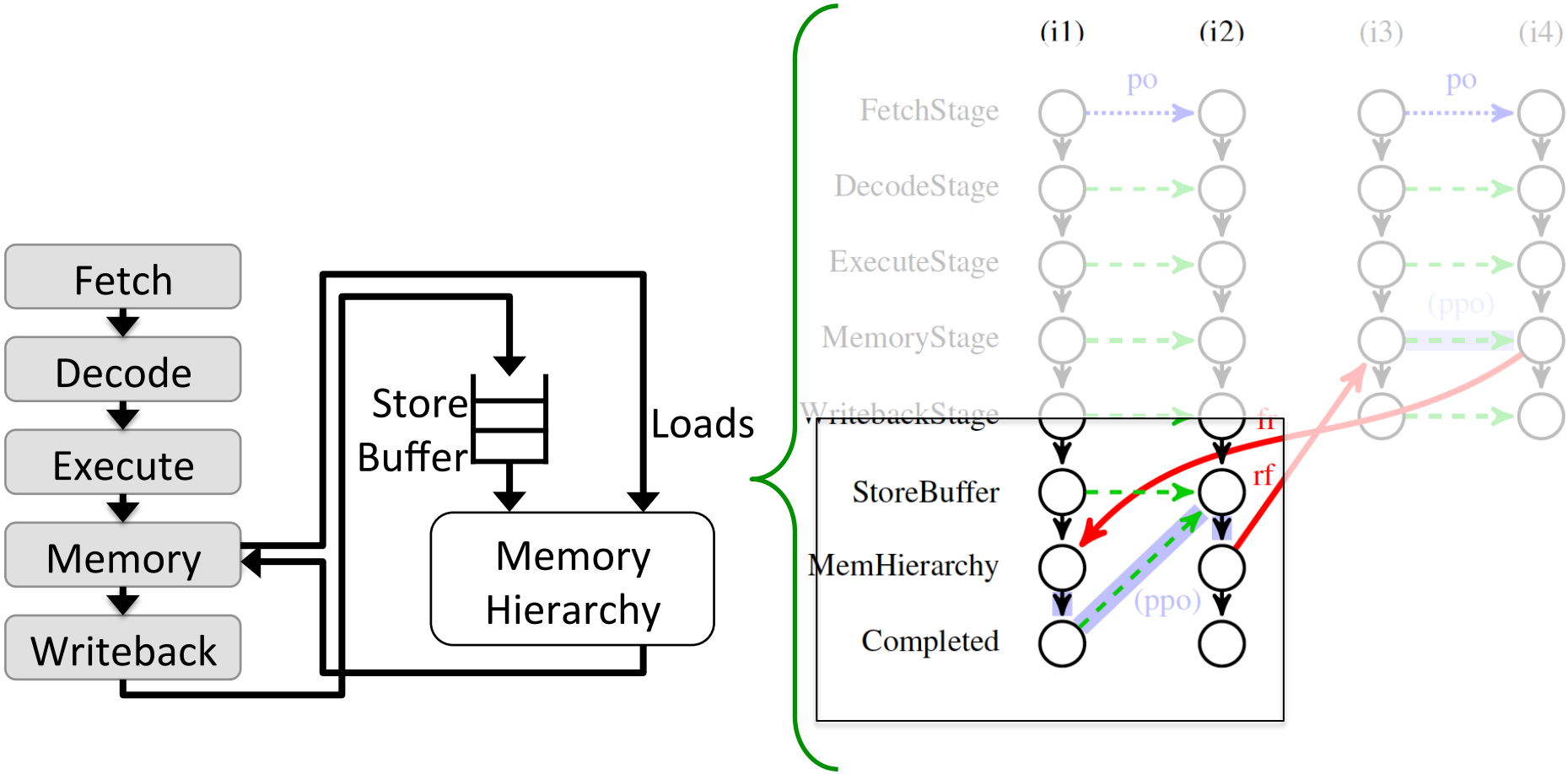
Reading From Particular Stages



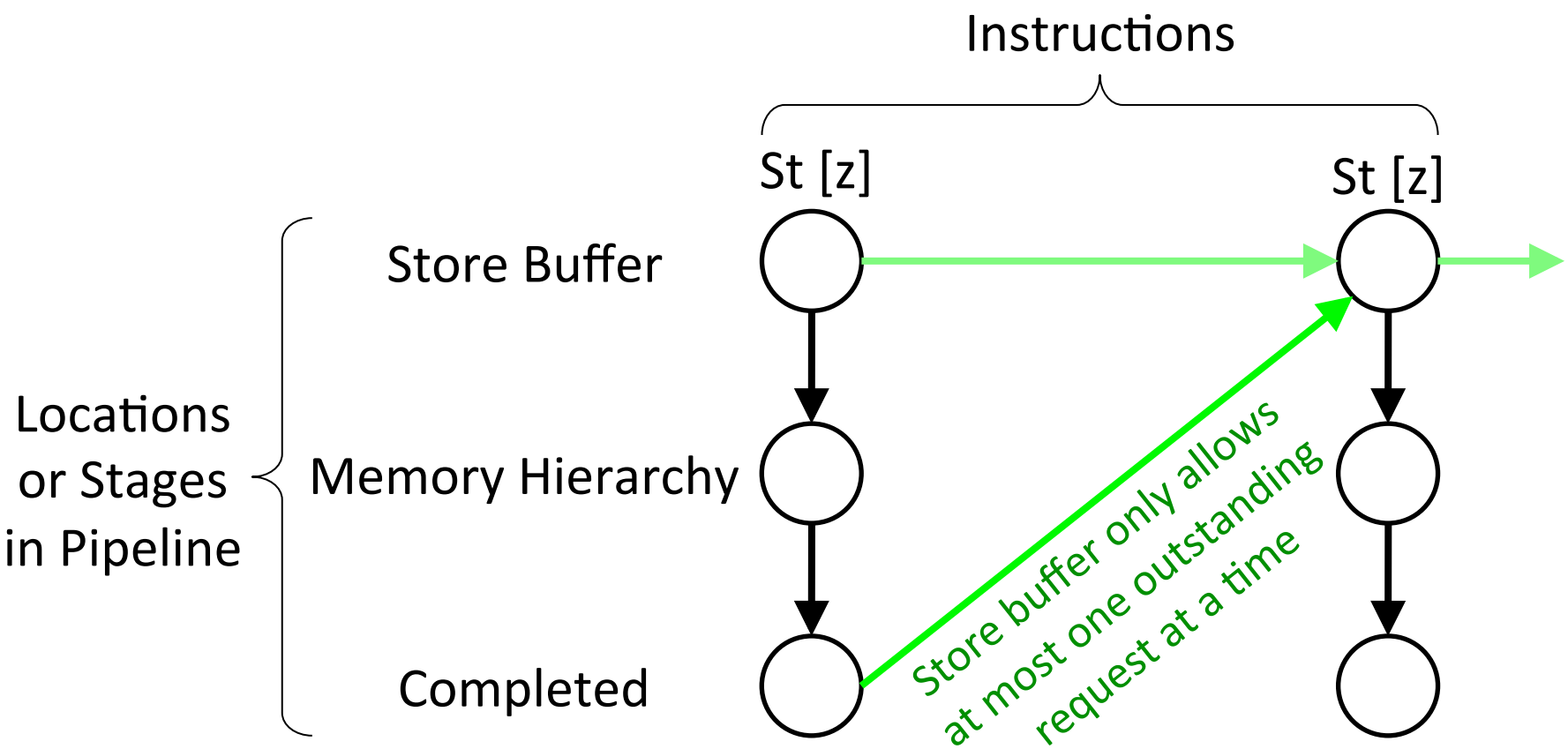
Reading From Particular Stages



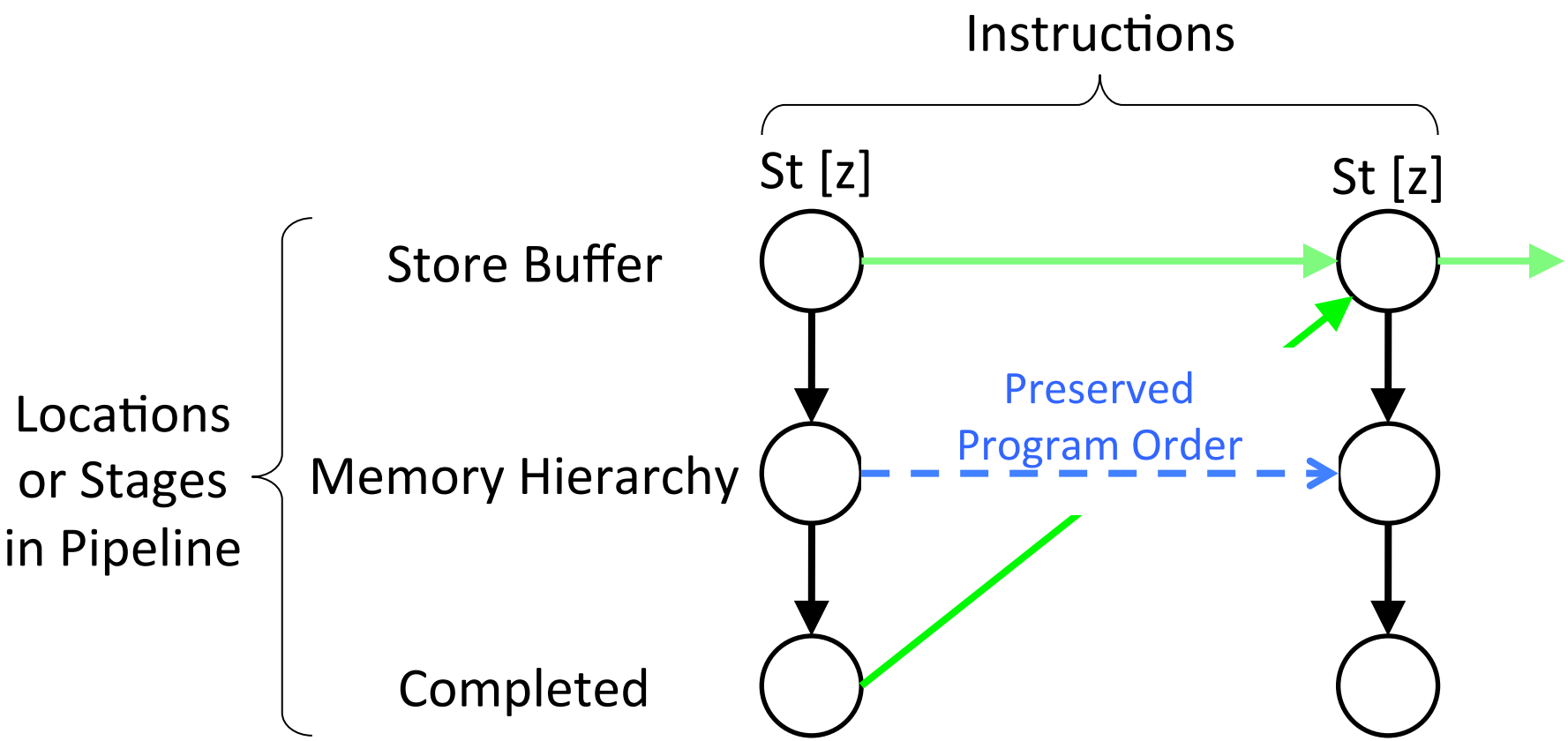
Microarchitectural Happens-Before Graphs



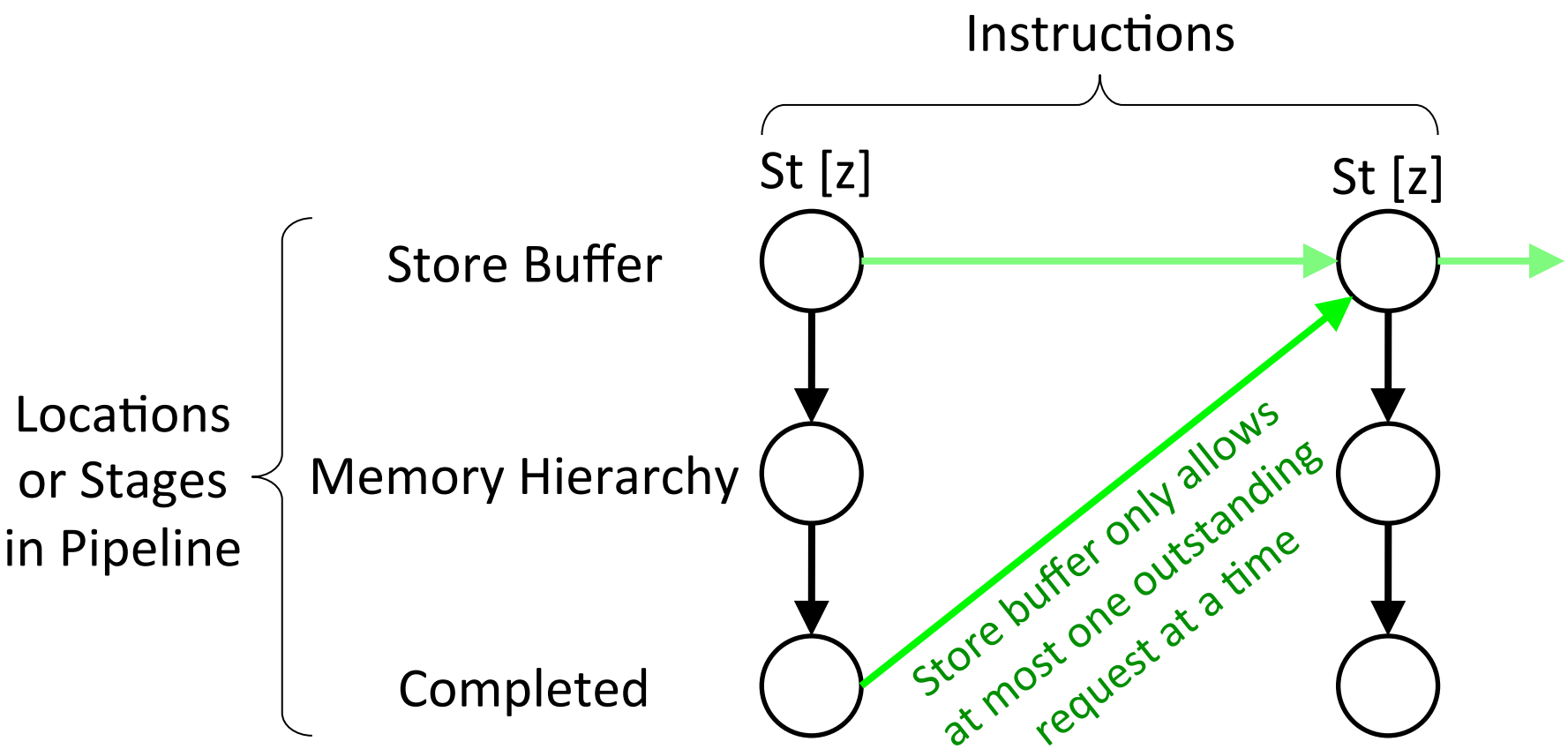
Verifying Preserved Program Order



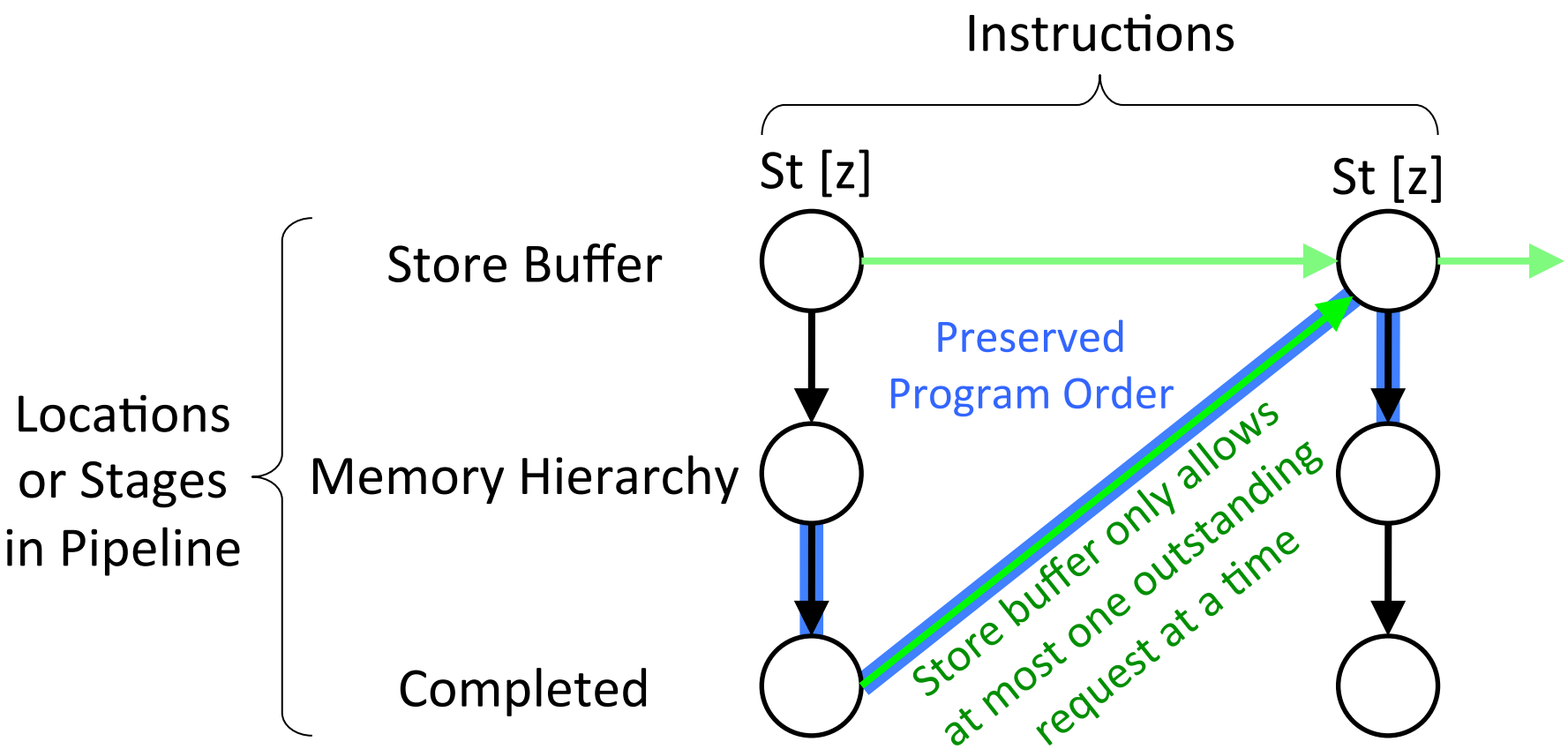
Verifying Preserved Program Order



Verifying Preserved Program Order



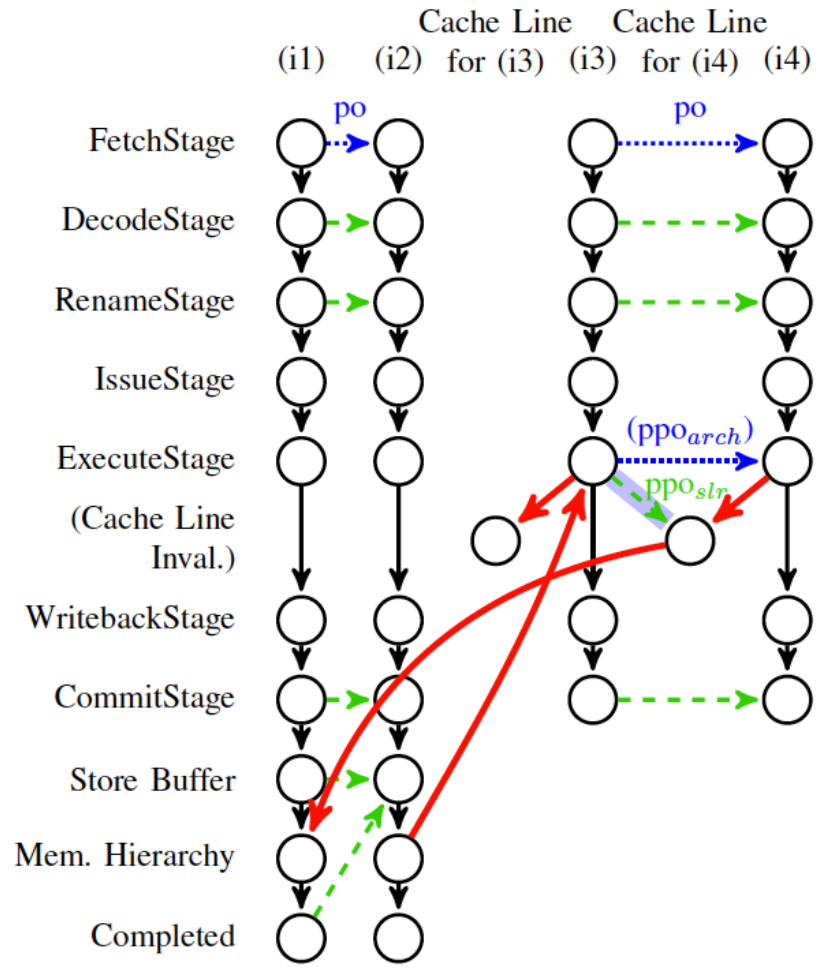
Verifying Preserved Program Order



Microarchitectural Optimizations

PipeCheck also supports:

- OoO cores
- Speculative load reordering
- WeeFence
- Heterogeneity
- and so on



PipeCheck Software Implementation

- Tool written in Coq and extracted to OCaml
- PipeCheck software verifies each pipeline against suite of litmus tests and PPO tests
  - **Automatically** enumerate all possible executions (i.e. all possible graphs) for each pipeline model/ test pair

	Observable	Not Observable
Permitted	OK	OK (µarch stricter than necessary)
Forbidden	Pipeline bug!!	OK

PipeCheck Pipeline Models

- Pipelines modeled by specifying:
  - list of stages
  - list of possible paths through the pipeline
  - set of "non-local edges" (details in paper)
  - list of "performing locations"

Pipeline	Lines of Code
Classic 5-Stage Pipeline without Store Buffer	37
Classic 5-Stage Pipeline with Store Buffer	62
gem5 O3 CPU Model	106
OpenSPARC T2	115

PipeCheck Litmus Test Results

Litmus Test	Expected	5-Stage w/o St. Buf	5-Stage w/ St. Buf	gem5 O3	OpenSPARC T2
iwp2.1/amd1	Forbid	✔	✔	Observed[2]	✔
iwp2.2/amd2	Forbid	✔	✔	✔	✔
iwp2.3a/amd4	Permit	Not obs.[1]	✔	✔	✔
iwp2.3b	Permit	✔	✔	✔	✔
iwp2.4/amd9	Permit	Not obs.[1]	✔	✔	✔
iwp2.5/amd8	Forbid	✔	✔	Observed[2]	✔
iwp2.6	Forbid	✔	✔	✔	✔
amd3	Permit	Not obs.[1]	✔	✔	✔
amd6	Forbid	✔	✔	Observed[2]	✔
n1	Permit	Not obs.[1]	✔	✔	✔
n2	Forbid	✔	✔	Observed[2]	✔
n4	Forbid	✔	✔	✔	✔
n5	Forbid	✔	✔	✔	✔
n6	Permit	✔	✔	✔	✔
n7	Permit	Not obs.[1]	✔	✔	✔
rwc-unfenced	Permit	Not obs.[1]	✔	✔	✔

1. Permitted results not observable: pipeline stronger than necessary

2. Forbidden results observable: found bugs in pipeline!

PipeCheck Litmus Test Results

Litmus Test	Expected	5-Stage w/o St. Buf	5-Stage w/ St. Buf	gem5 O3	OpenSPARC T2
iwp2.1/amd1	Forbid	✔	✔	**Observed[2]**	✔
iwp2.2/amd2	Forbid	✔	✔	✔	✔
iwp2.3a/amd4	Permit	**Not obs.[1]**	✔	✔	✔
iwp2.3b	Permit	✔	✔	✔	✔
iwp2.4/amd9	Permit	**Not obs.[1]**	✔	✔	✔
iwp2.5/amd8	Forbid	✔	✔	**Observed[2]**	✔
iwp2.6	Forbid	✔	✔	✔	✔
amd3	Permit	**Not obs.[1]**	✔	✔	✔
amd6	Forbid	✔	✔	**Observed[2]**	✔
n1	Permit	**Not obs.[1]**	✔	✔	✔
n2	Forbid	✔	✔	**Observed[2]**	✔
n4	Forbid	✔	✔	✔	✔
n5	Forbid	✔	✔	✔	✔
n6	Permit	✔	✔	✔	✔
n7	Permit	**Not obs.[1]**	✔	✔	✔
rwc-unfenced	Permit	**Not obs.[1]**	✔	✔	✔

1. Permitted results not observable: pipeline stronger than necessary

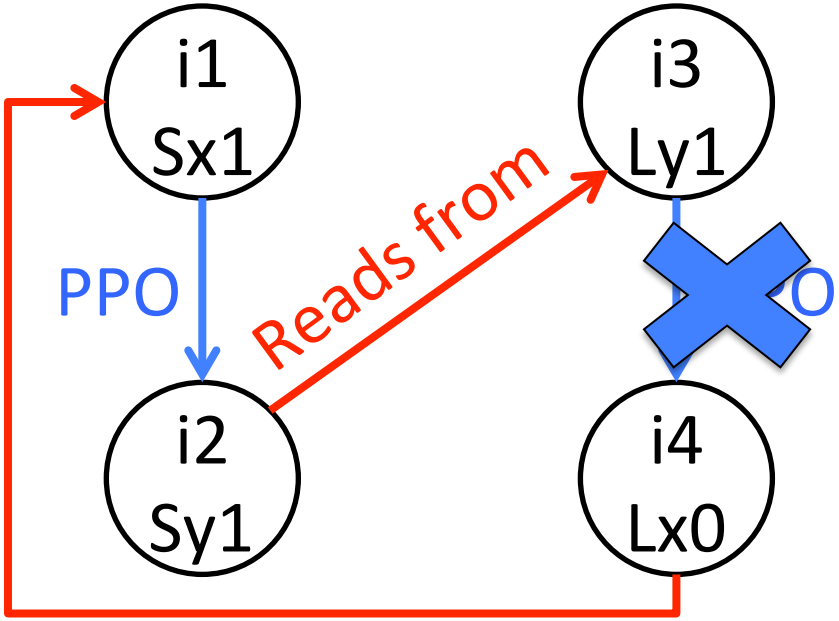
2. Forbidden results observable: found bugs in pipeline!

Interpreting Bugs Found By PipeCheck

Initially: [x]=[y]=0

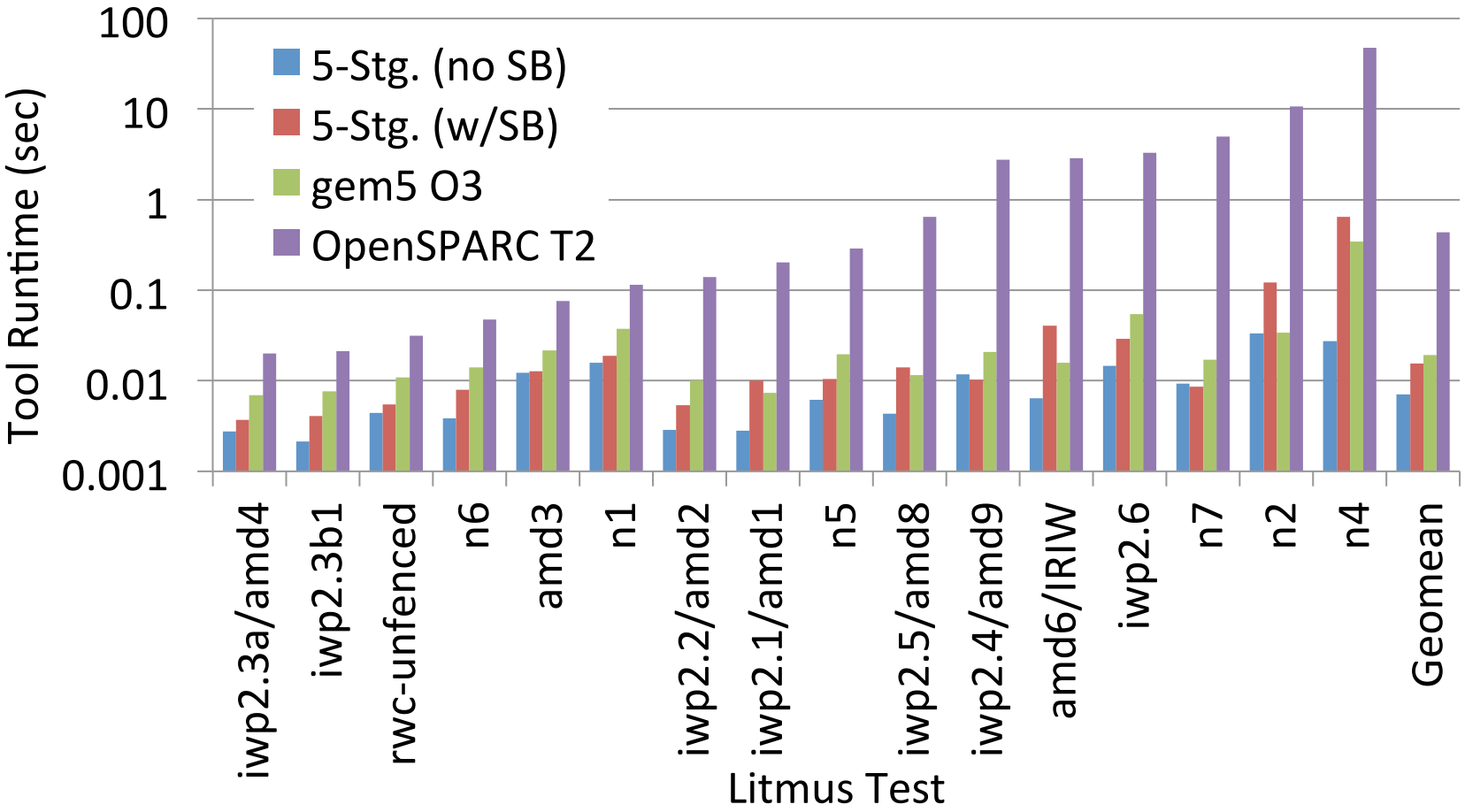
Core 0	Core 1
(i1) st [x], 1 (i2) st [y], 1	(i3) ld [y] → r1 (i4) ld [x] → r2

TSO: **Forbid**: r1=1, r2=0



Reads from earlier store
("from-reads")

PipeCheck Verification Time



PipeCheck Conclusions

- PipeCheck verifies the memory model ***implementation*** against vendor spec
  - Automated tool with practical runtimes
- Opens door to systematized checking of heterogeneous mix of memory referencers: CPU, GPU, accelerators, or anything else

PipeCheck: Specifying and Verifying Microarchitectural Enforcement of Memory Consistency Models

Daniel Lustig[+], Michael Pellauer[*],
and Margaret Martonosi[+]

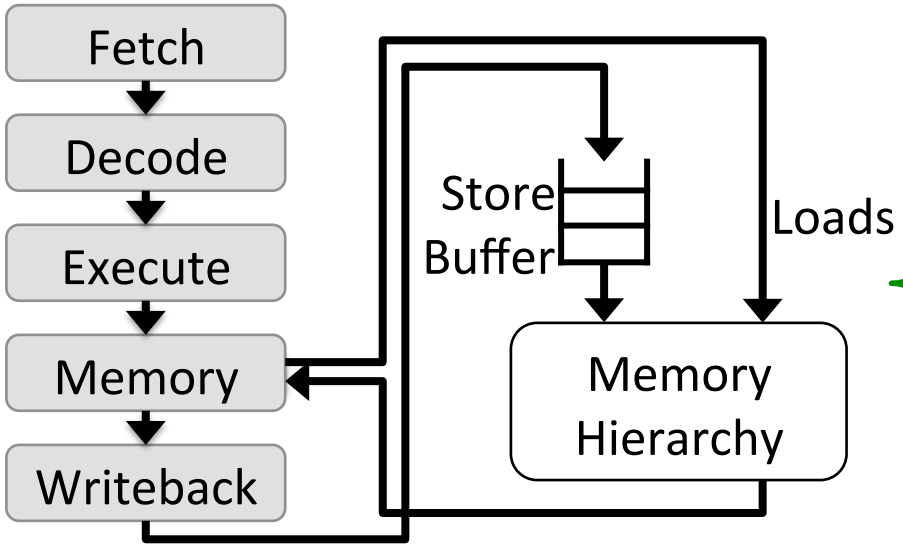
[+]Princeton University [*]Intel VSSAD

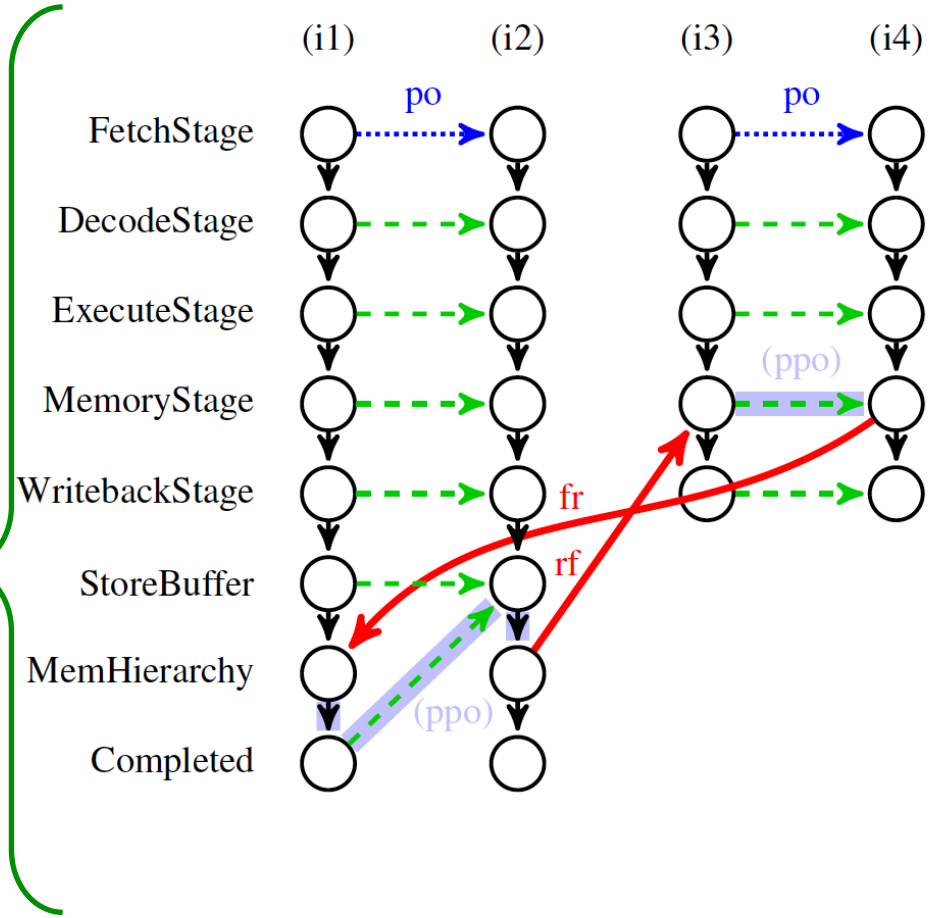
MICRO 47

PRINCETON

PipeCheck Overview

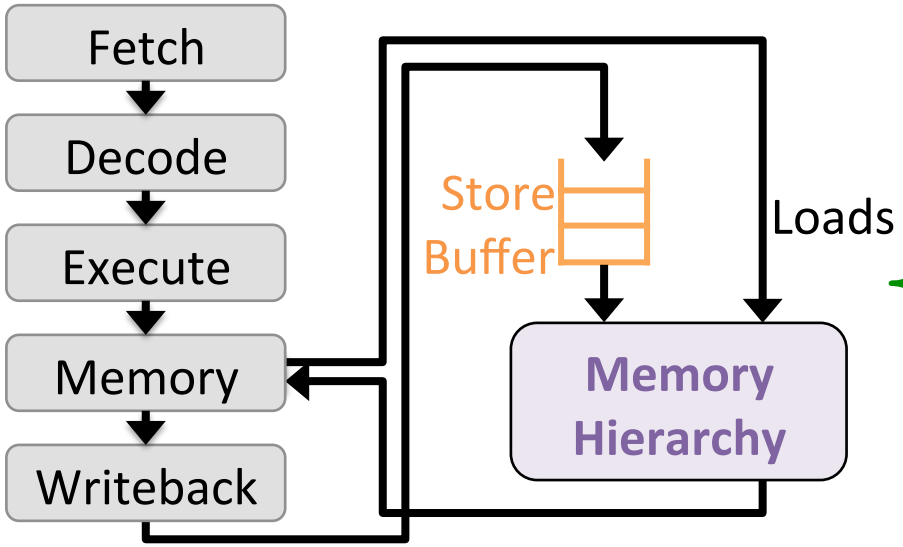
- **Microarchitectural equivalent** of "happens-before" graphs

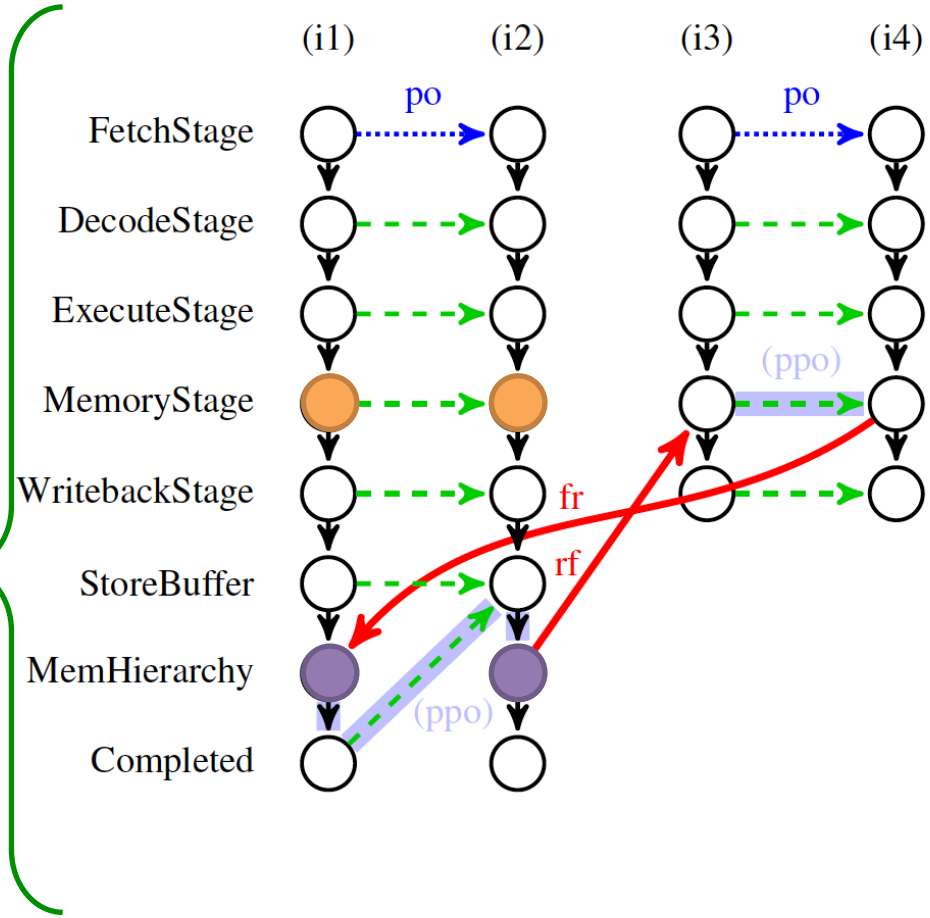




PipeCheck Overview

- **Microarchitectural equivalent** of "happens-before" graphs



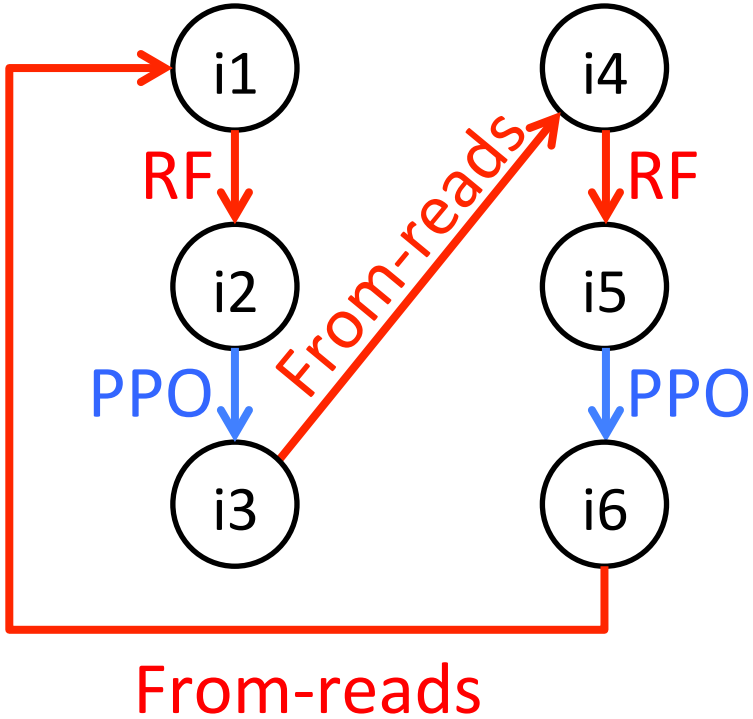


Non-SC Models: Not Every Cycle Forbids a Proposed Outcome

Litmus test iwp2.4/amd9:

Core 0	Core 1
(i1) st [x], 1	(i4) st [y], 1
(i2) ld [x] → r1	(i5) ld [y] → r3
(i3) ld [y] → r2	(i6) ld [x] → r4

Outcome: r1=r3=1, r2=r4=0
Observed on Hardware



[Intel, AMD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

More Complexity = Special Cases

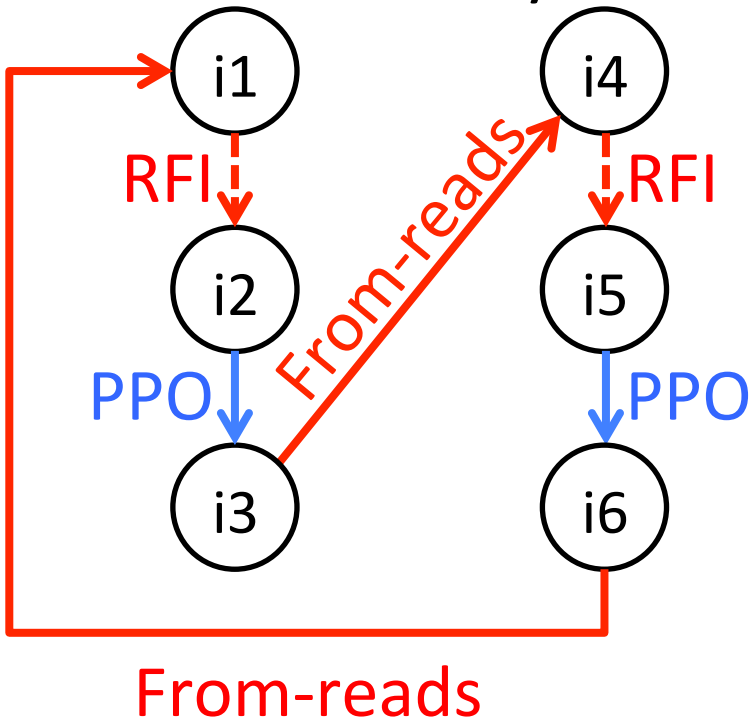
Litmus test iwp2.4/amd9:

Core 0	Core 1
(i1) st [x], 1	(i4) st [y], 1
(i2) ld [x] → r1	(i5) ld [y] → r3
(i3) ld [y] → r2	(i6) ld [x] → r4

TSO: **Allow** r1=r3=1, r2=r4=0

To fully capture all model details, need either complex nodes or complex **edges**

Architectural Analysis:



[Alglave, FMSD '12, Owens et al., TPHOLs '09]

More Complexity = Special Cases

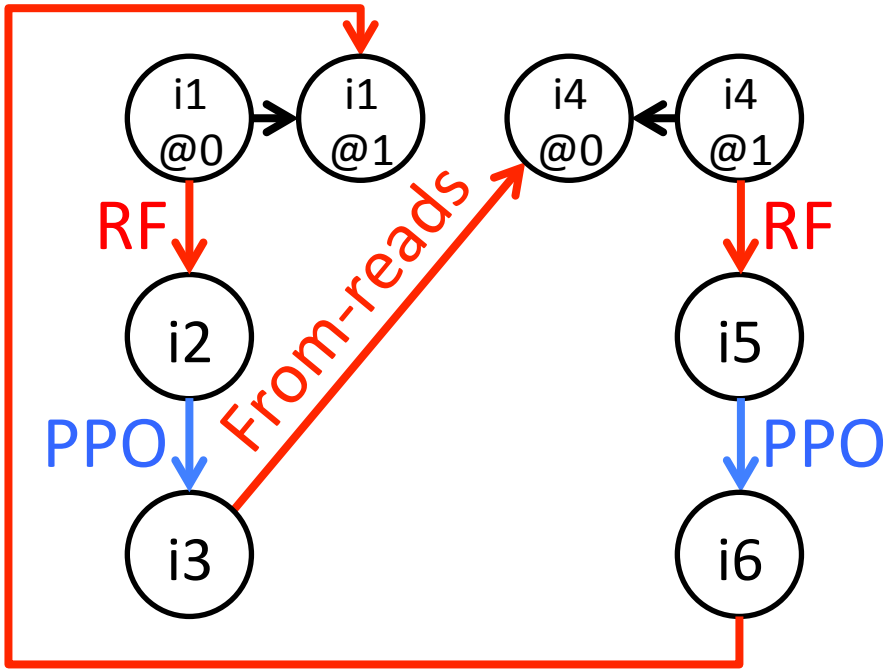
Litmus test iwp2.4/amd9:

Core 0	Core 1
(i1) st [x], 1 (i2) ld [x] → r1 (i3) ld [y] → r2	(i4) st [y], 1 (i5) ld [y] → r3 (i6) ld [x] → r4

TSO: **Allow** r1=r3=1, r2=r4=0

To fully capture all model details, need either complex **nodes** or complex edges

Architectural Analysis:



From-reads

What is the Definition of "Correct"?

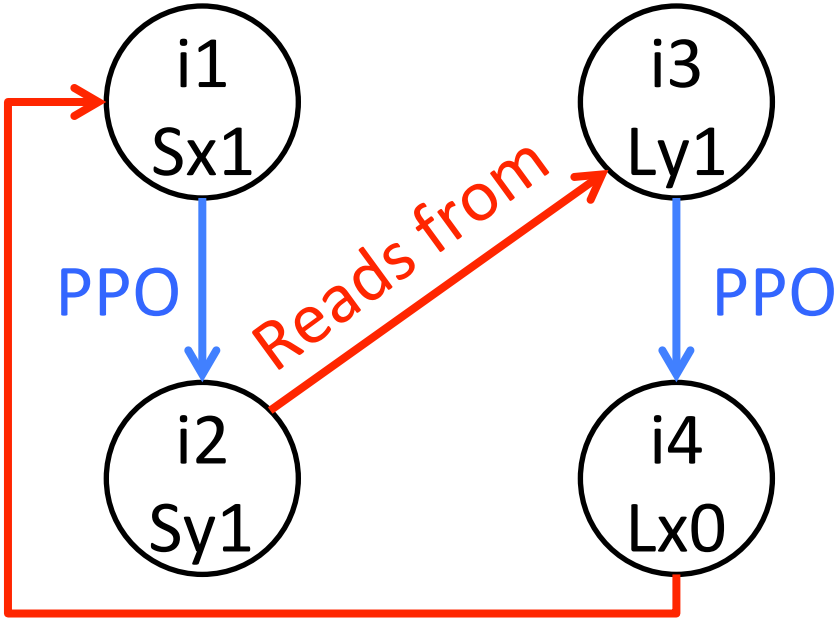
Litmus test mp:

Initially: [x]=[y]=0

Core 0	Core 1
(i1) st [x], 1 (i2) st [y], 1	(i3) ld [y] → r1 (i4) ld [x] → r2

TSO: **Forbid**: r1=1, r2=0

Cycle implies forbidden execution: instruction can't happen before itself

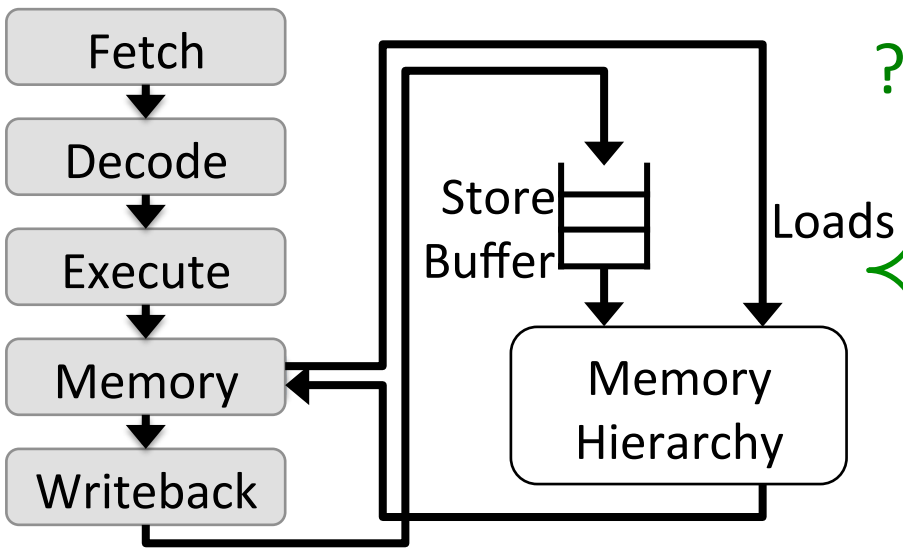


Reads from earlier store ("from-reads")

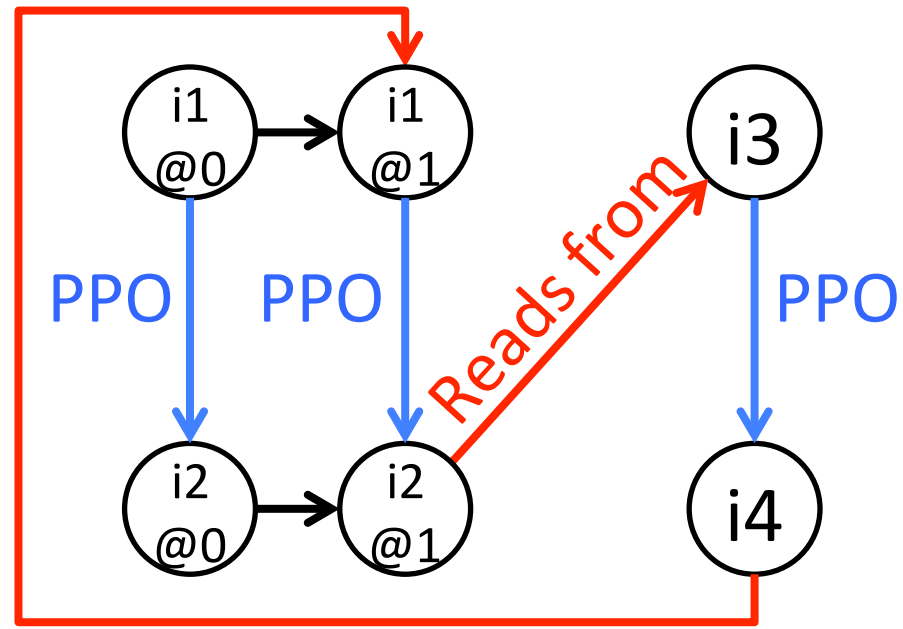
[MD, Alglave, FMSD '12, Owens et al., TPHOLs '09]

Insights from Microarchitecture

- How is the consistency model **enforced at the microarchitecture level**?



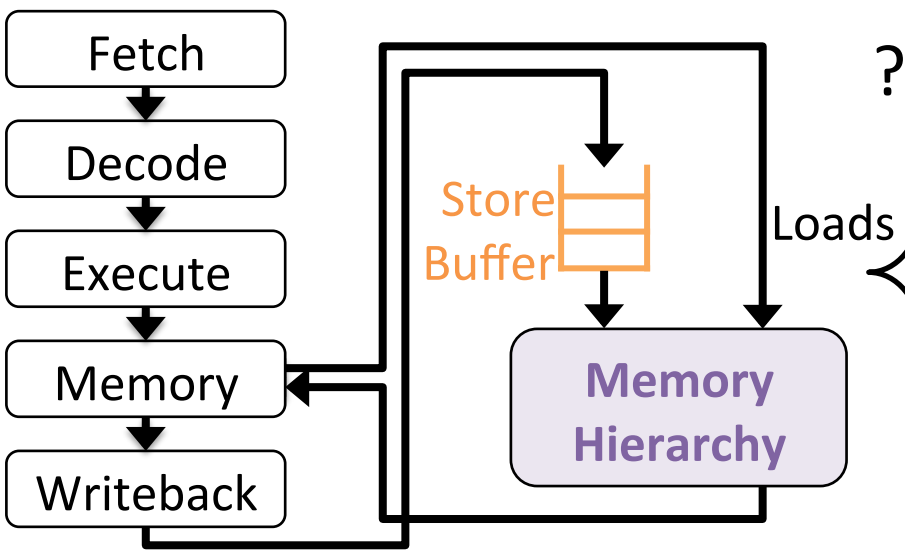
?



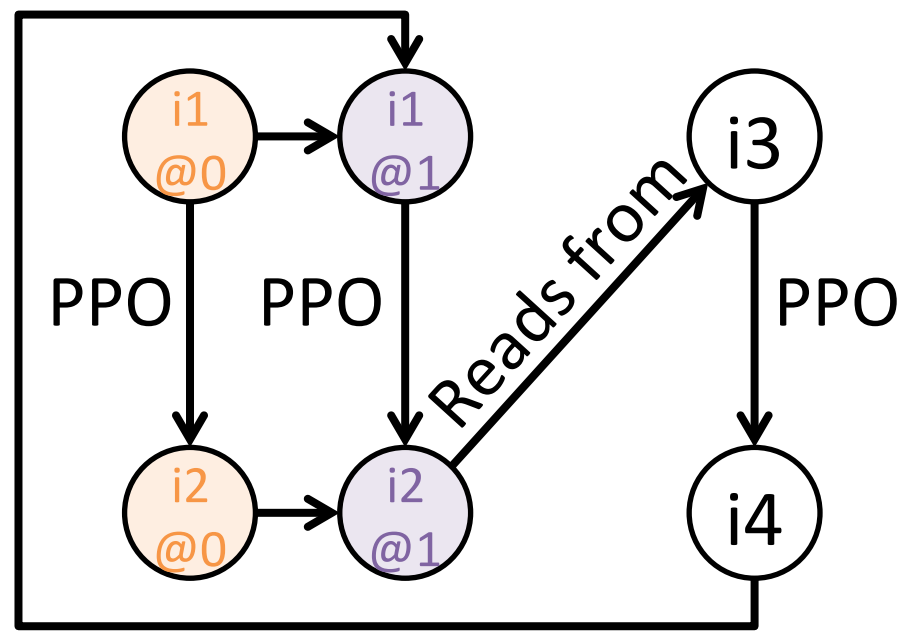
Reads from earlier ("from-reads")

Insights from Microarchitecture

- How is the consistency model **enforced at the microarchitecture level**?



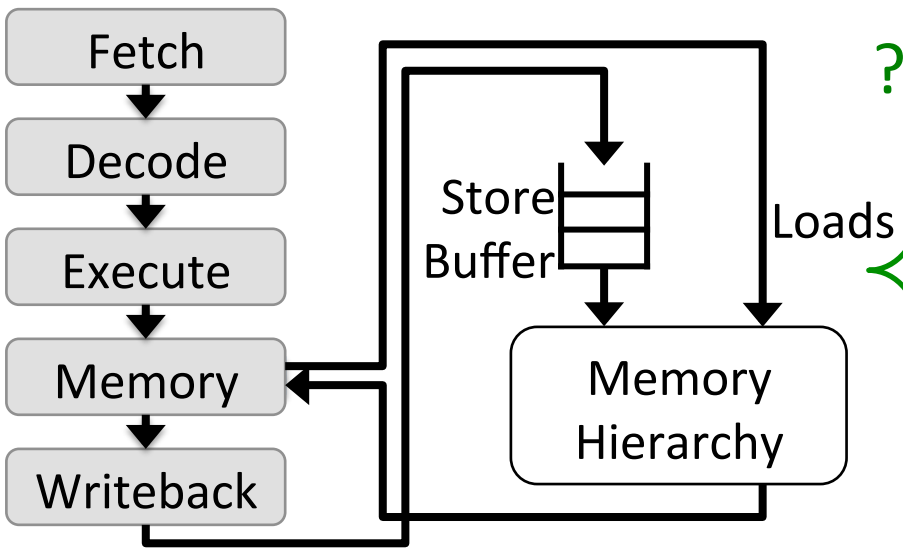
?



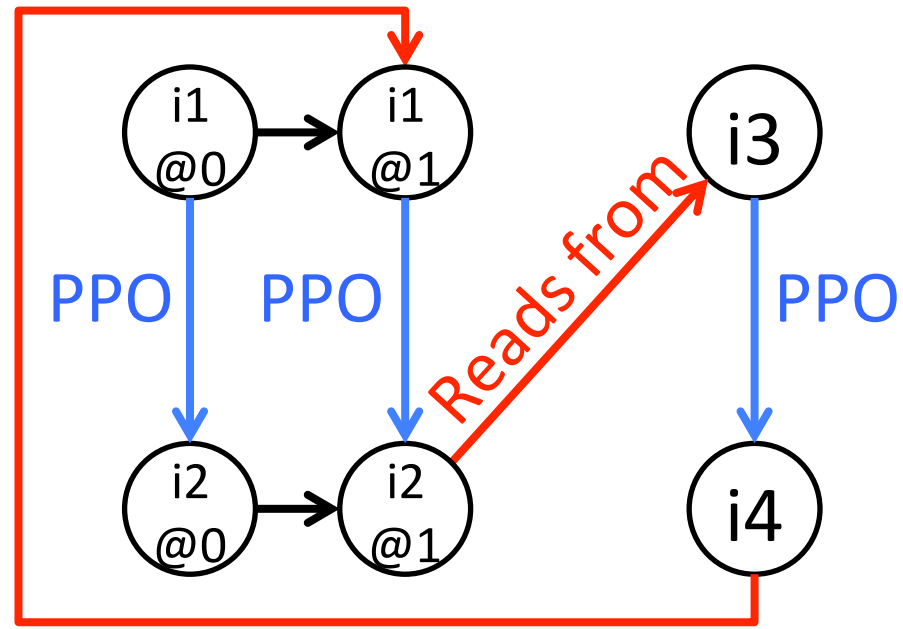
Reads from earlier ("from-reads")

Insights from Microarchitecture

- How is the consistency model **enforced at the microarchitecture level**?

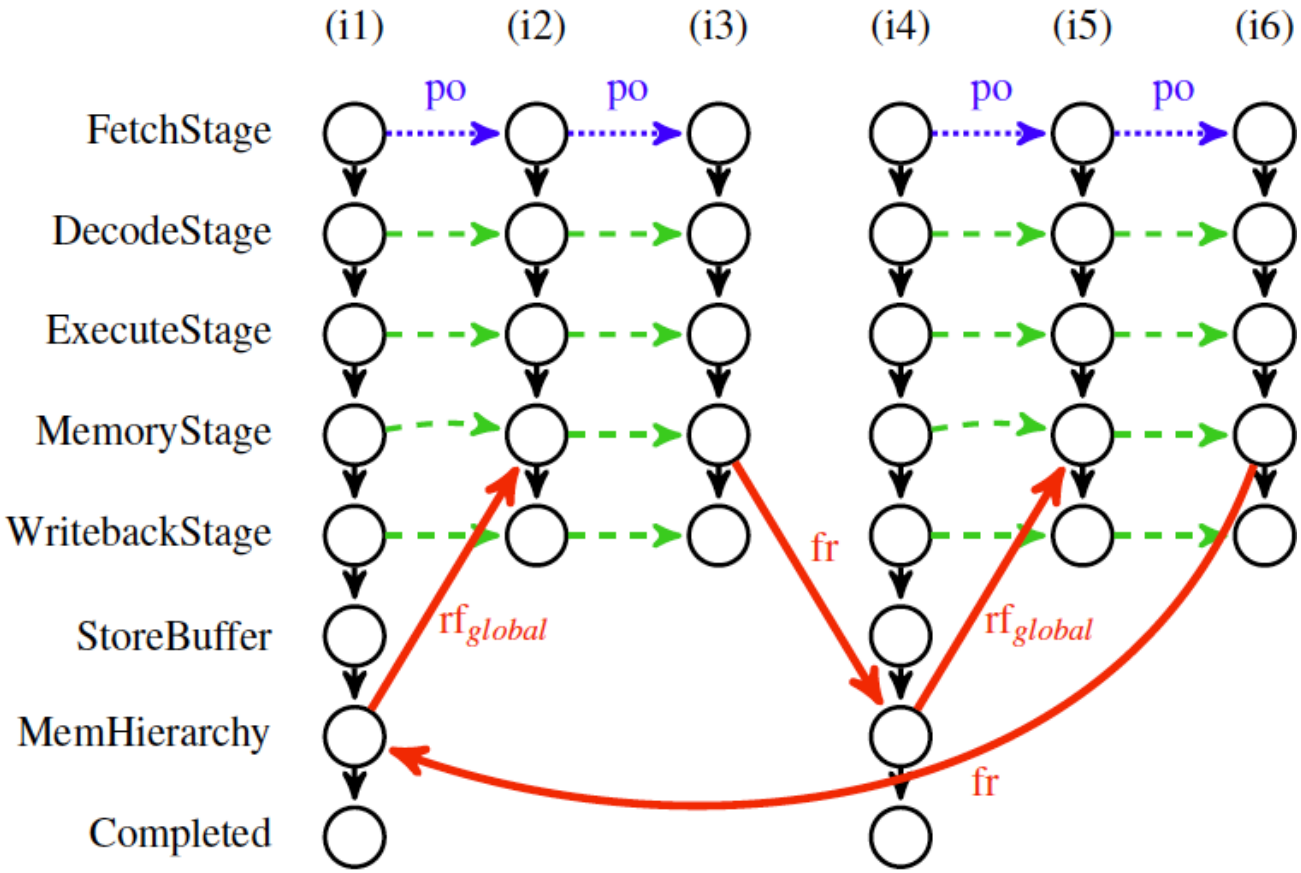


?



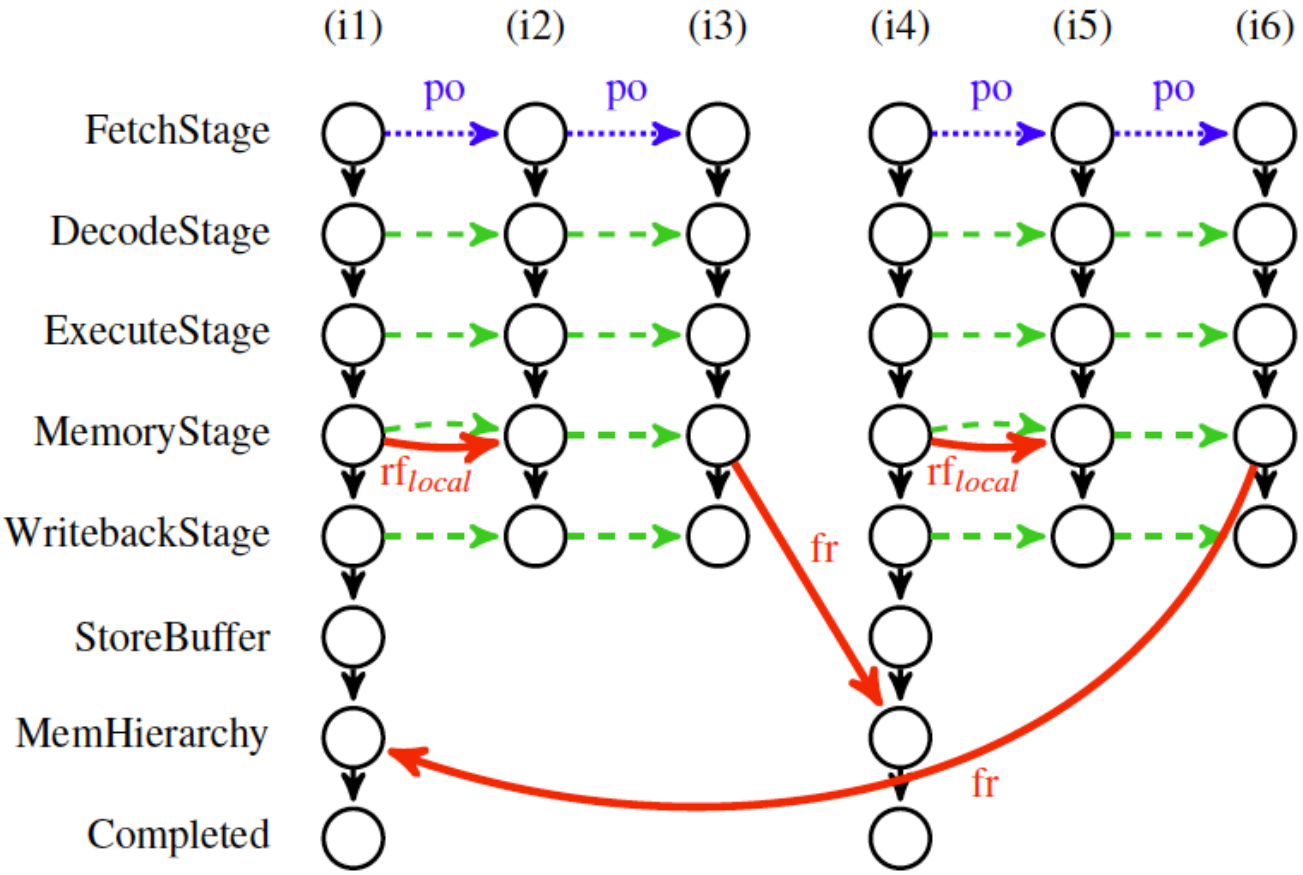
Reads from earlier ("from-reads")

At µarch Level, Cycle = Forbidden

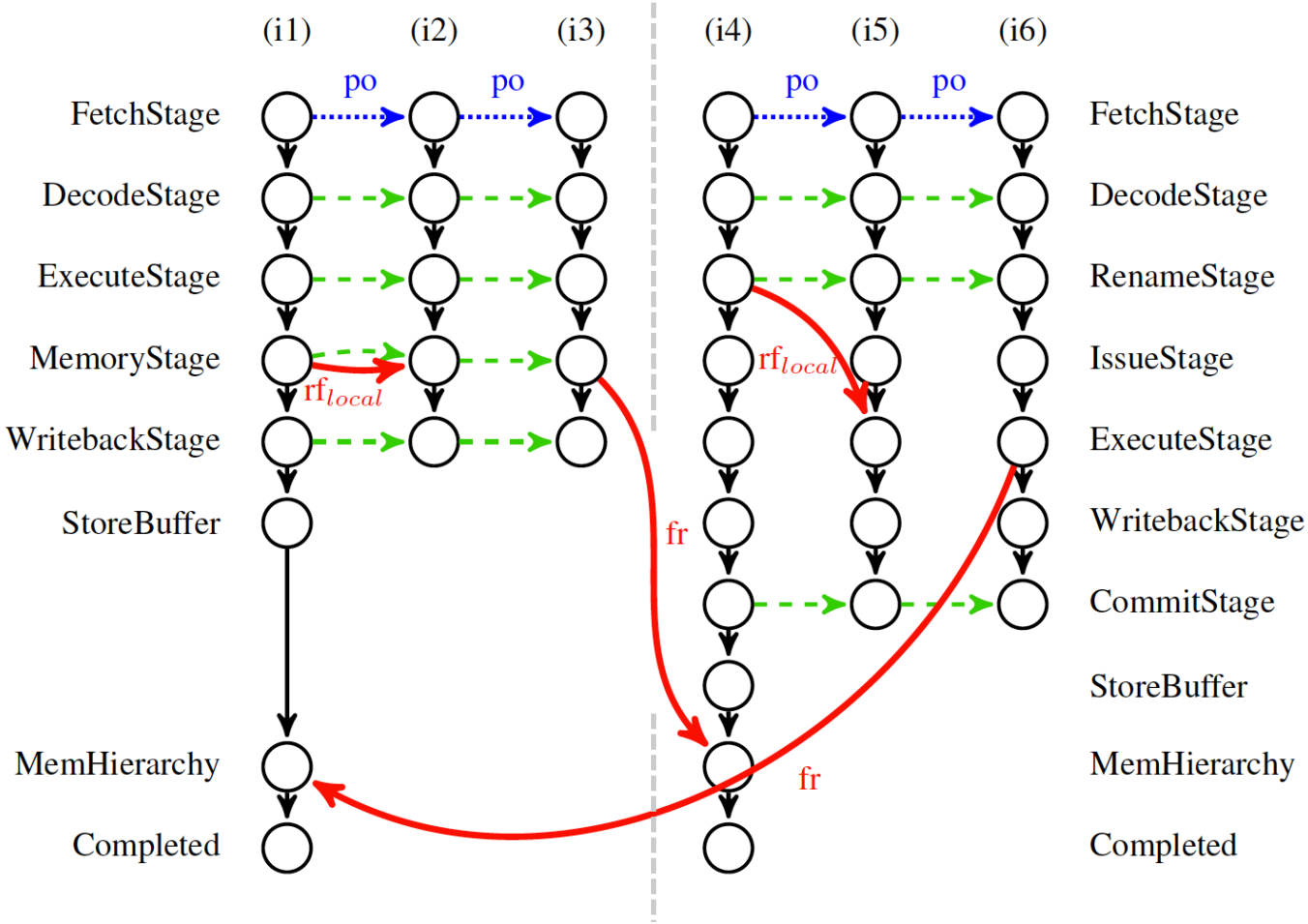


Satisfy reads from memory → µhb cycle → forbidden

At µarch Level, Cycle = Forbidden



Satisfy reads from store buffer → no µhb cycle → permitted

Supporting Heterogeneity

Different cores can have different µarches – no special support is needed